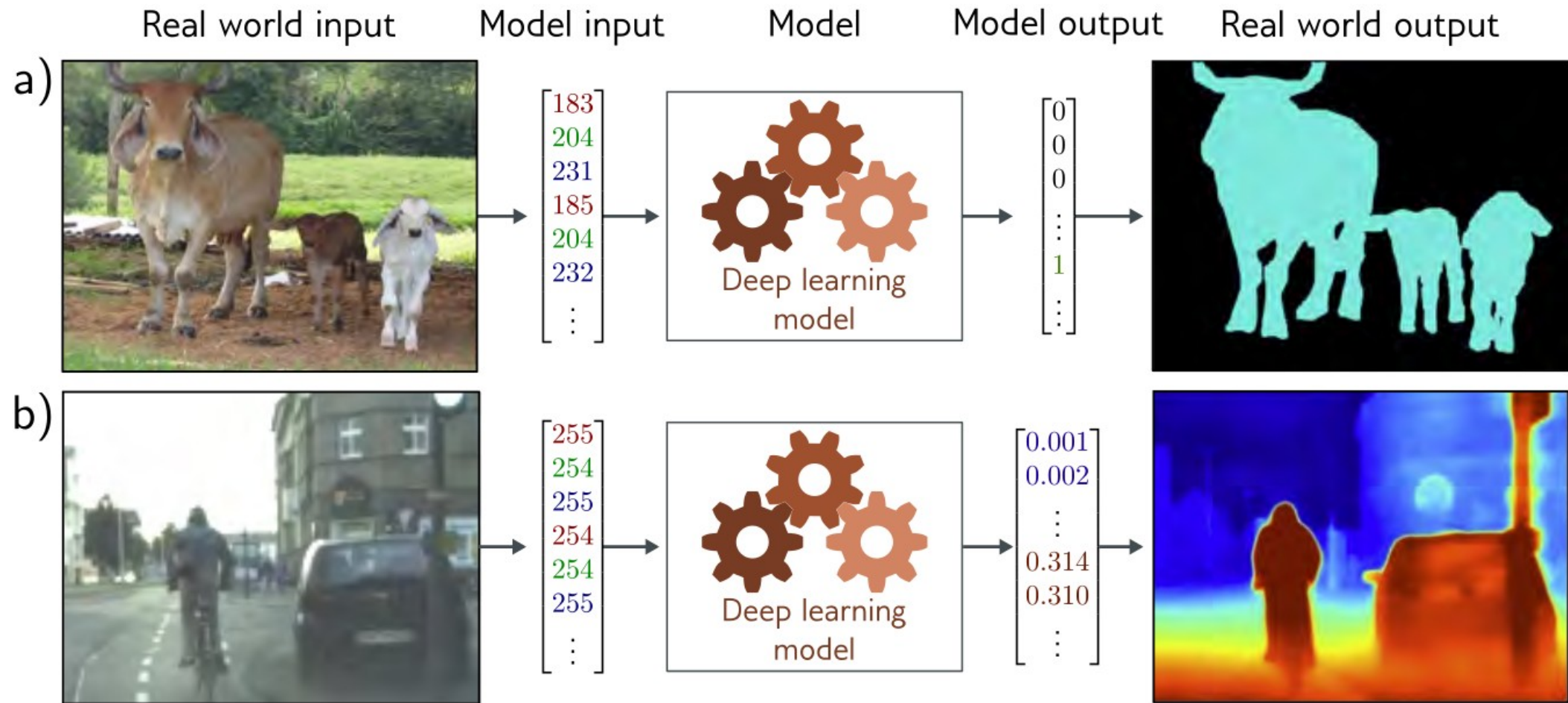


Automatic Image Analysis

Berlin University of Technology (TUB),
Computer Vision and Remote Sensing Group
Berlin, Germany

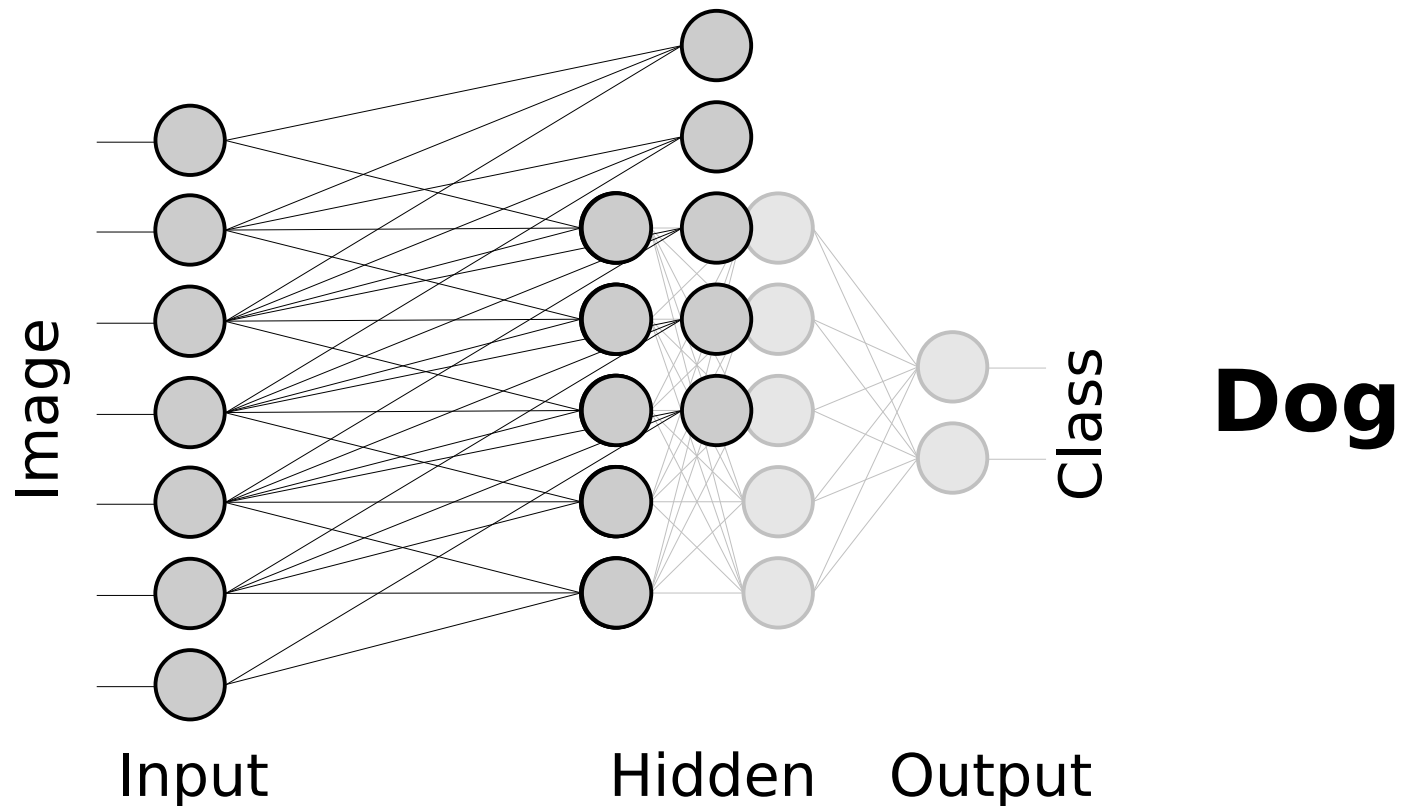


Image Analysis with Deep Learning Model

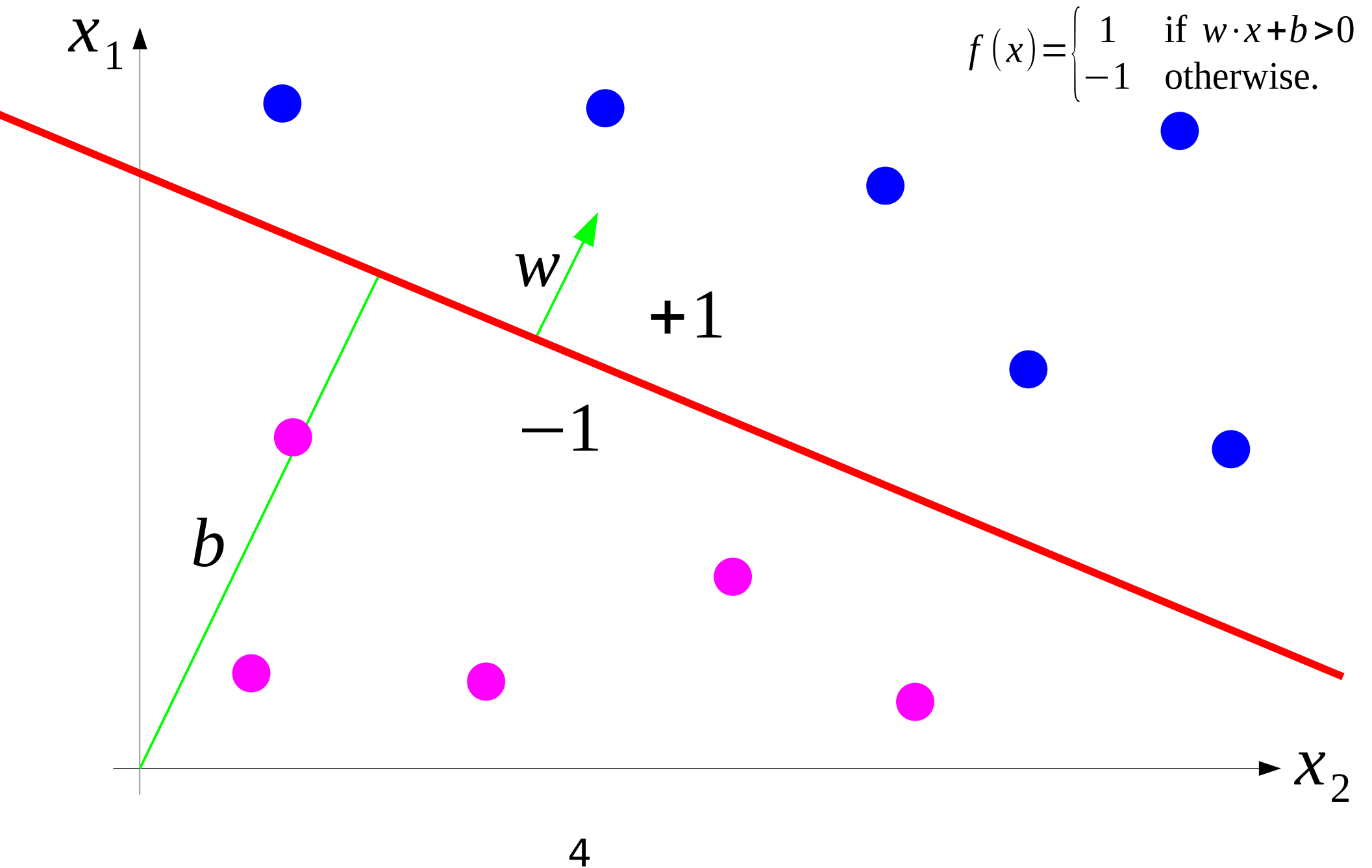


Analyzing an Image with an Artificial Neural Network

Multiple features
with different kernels

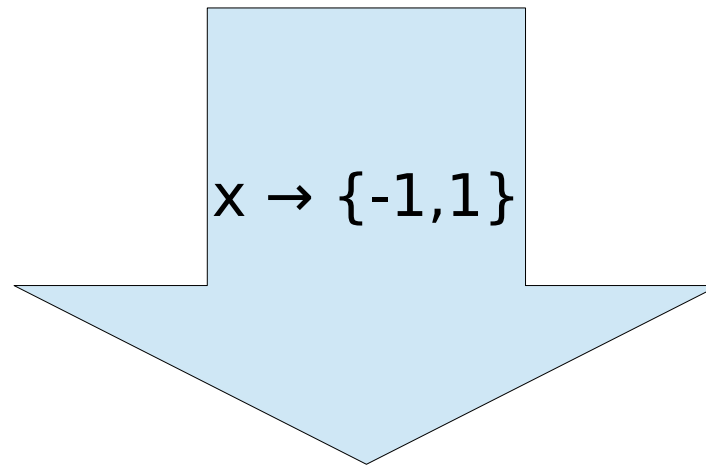


Perceptron



Perceptron

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ -1 & \text{otherwise.} \end{cases}$$



$$f(x) = \text{sign}(w \cdot x)$$

Perceptron

$$f(x) = \text{sign}(w \cdot x)$$

1. Initialize weights w and bias b
(e.g. small random numbers)
2. For each training sample (x, y) :
 - 2.1. Calculate output $f(x)$
 - 2.2. Update weights according to $w_i^{t+1} = w_i^t + (y - f(x)) x_i$
3. Go to 2 until convergence
e.g. average weight change smaller than a threshold

Linear classifier = **Linear** decision boundary

→ Converges, if training data linear separable

→ If not, training fails completely (in standard version)

There are problems that are not solvable by a perceptron

→ e.g. XOR

→ Solution: **Multi-layer Perceptron (MLP)**

Perceptron

$$f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x})$$

$$f(\mathbf{x}) \approx \mathbf{w} \cdot \mathbf{x}$$

1. Initialize weights $\mathbf{w}$ and bias b
(e.g. small random numbers)
2. For each training sample $(\mathbf{x}, y)$:
 - 2.1. Calculate output $f(\mathbf{x})$
 - 2.2. Update weights according to $\mathbf{w}_i^{t+1} = \mathbf{w}_i^t + \eta(y - f(\mathbf{x})) \mathbf{x}_i$
3. Go to 2 until convergence
e.g. average weight change smaller than a threshold

Linear classifier = **Linear** decision boundary

→ Converges if training data linear separable

→ If not, training fails completely (in standard version)

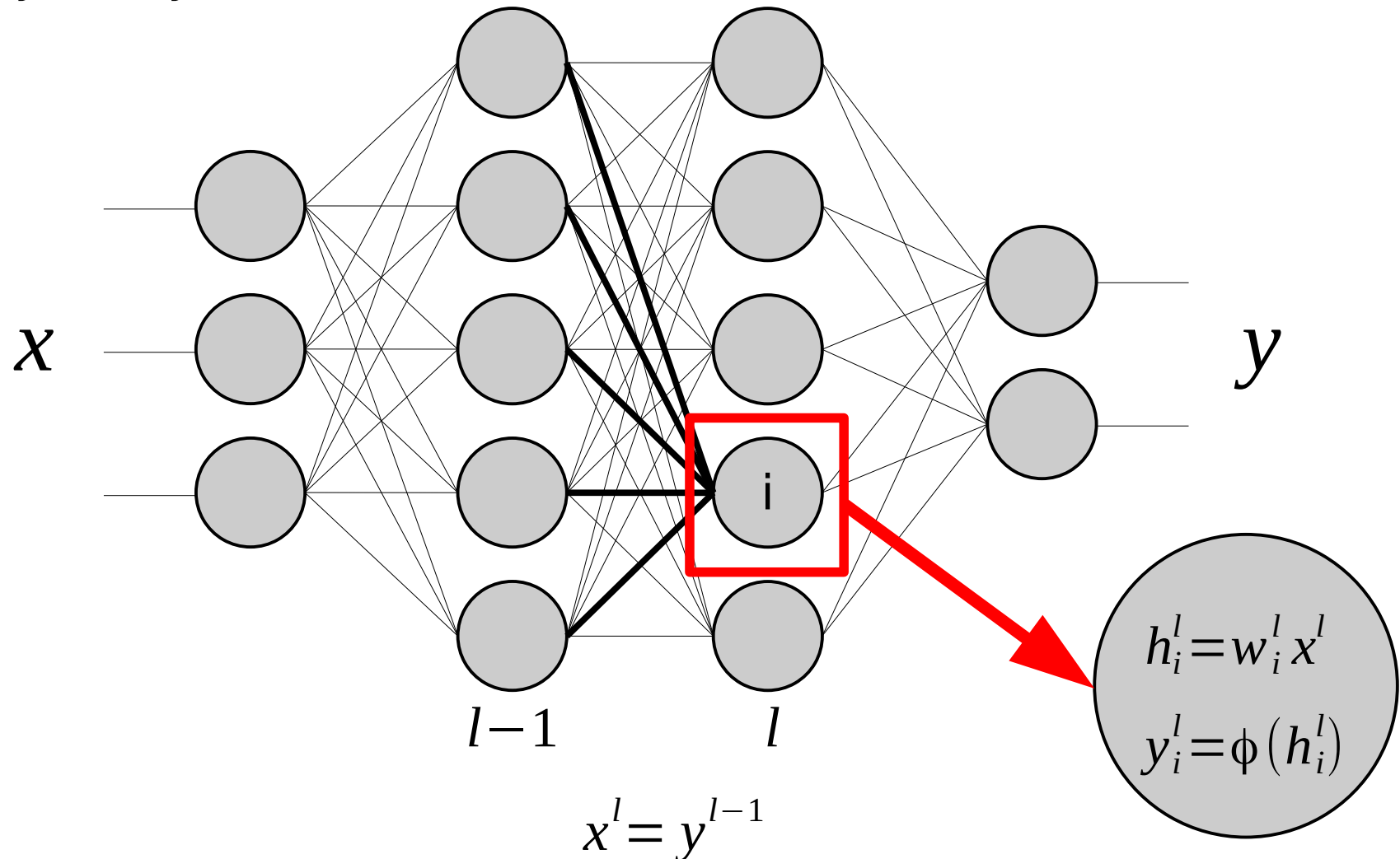
There are problems that are not solvable by a perceptron

→ e.g. XOR

→ Solution: **Multi-layer Perceptron (MLP)**

Multilayer Perceptron

- Feed-forward artificial neural network
- Multiple layers of simple units (neurons)
- Each layer fully connected to the next one without lateral connections



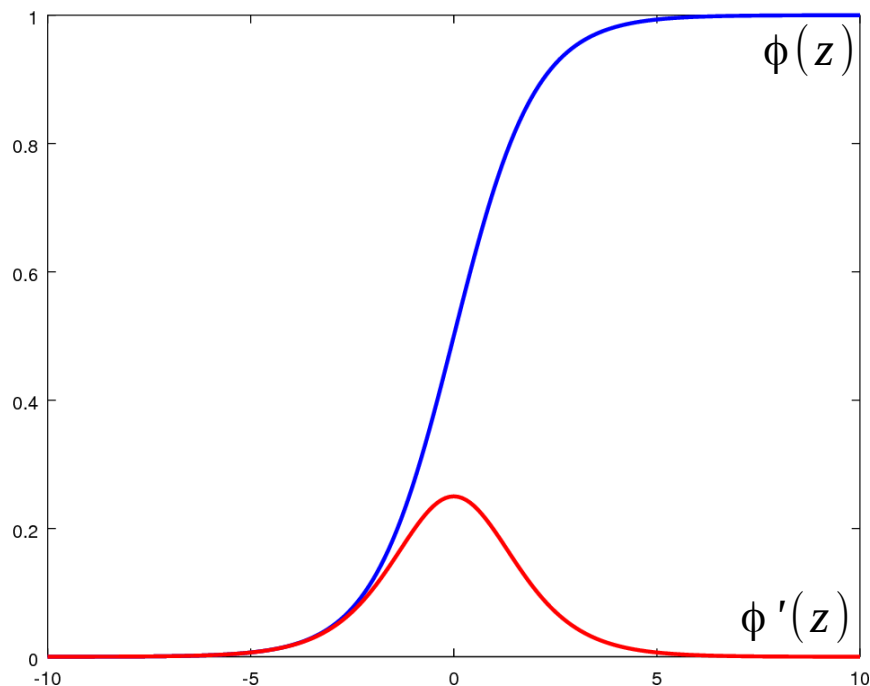
Activation Functions ϕ

- Usually non-linear, bounded, and analytical
- Common choices (for MLPs)

→ Logistic function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

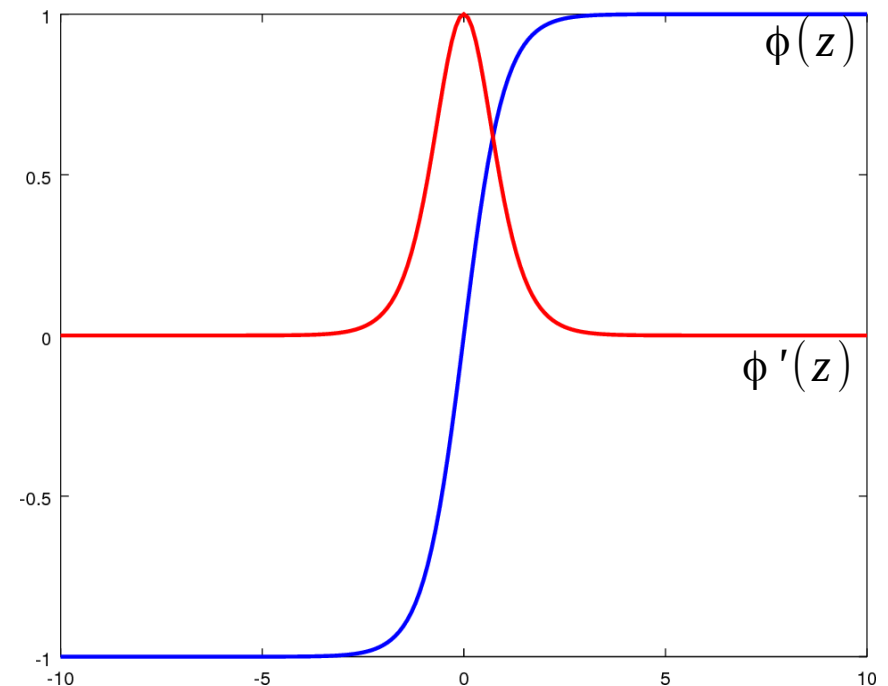
$$\phi'(z) = \phi(z)(1 - \phi(z))$$



→ Tangens hyperbolicus

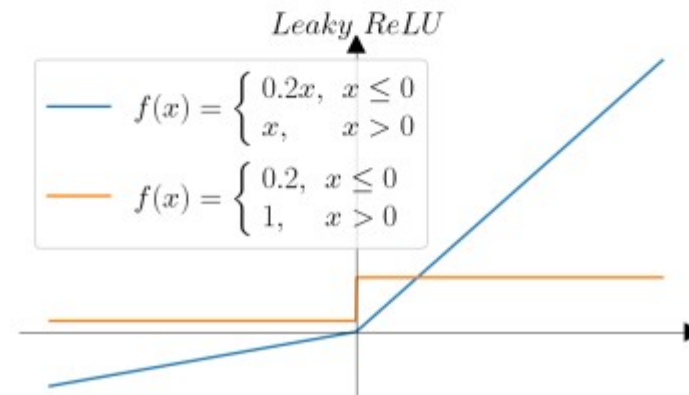
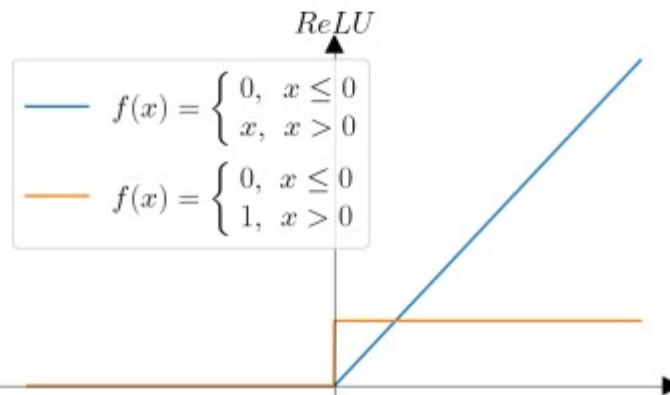
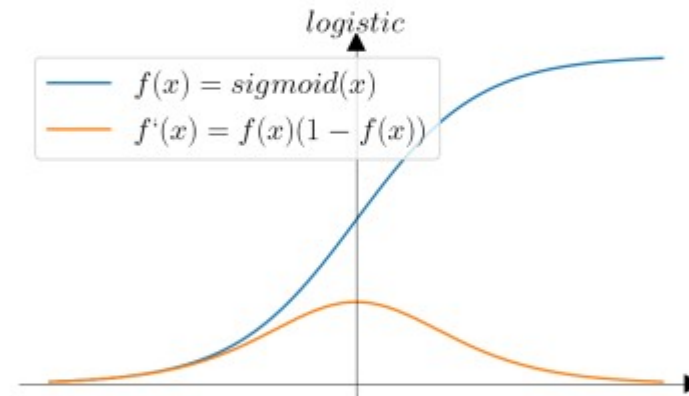
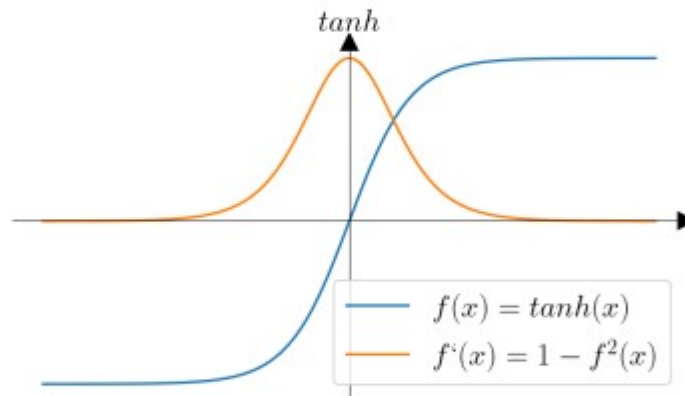
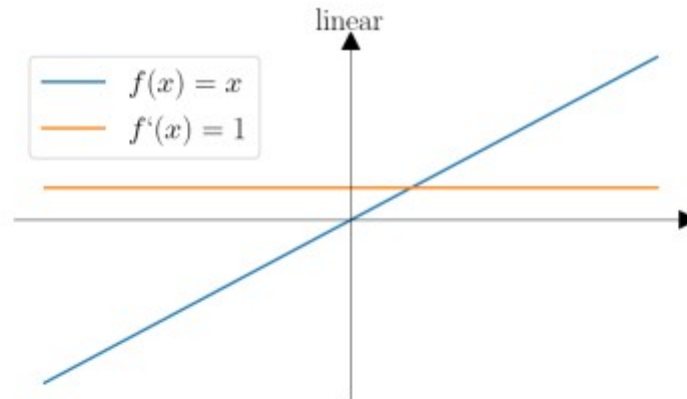
$$\phi(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\phi'(z) = 1 - \tanh^2(z)$$

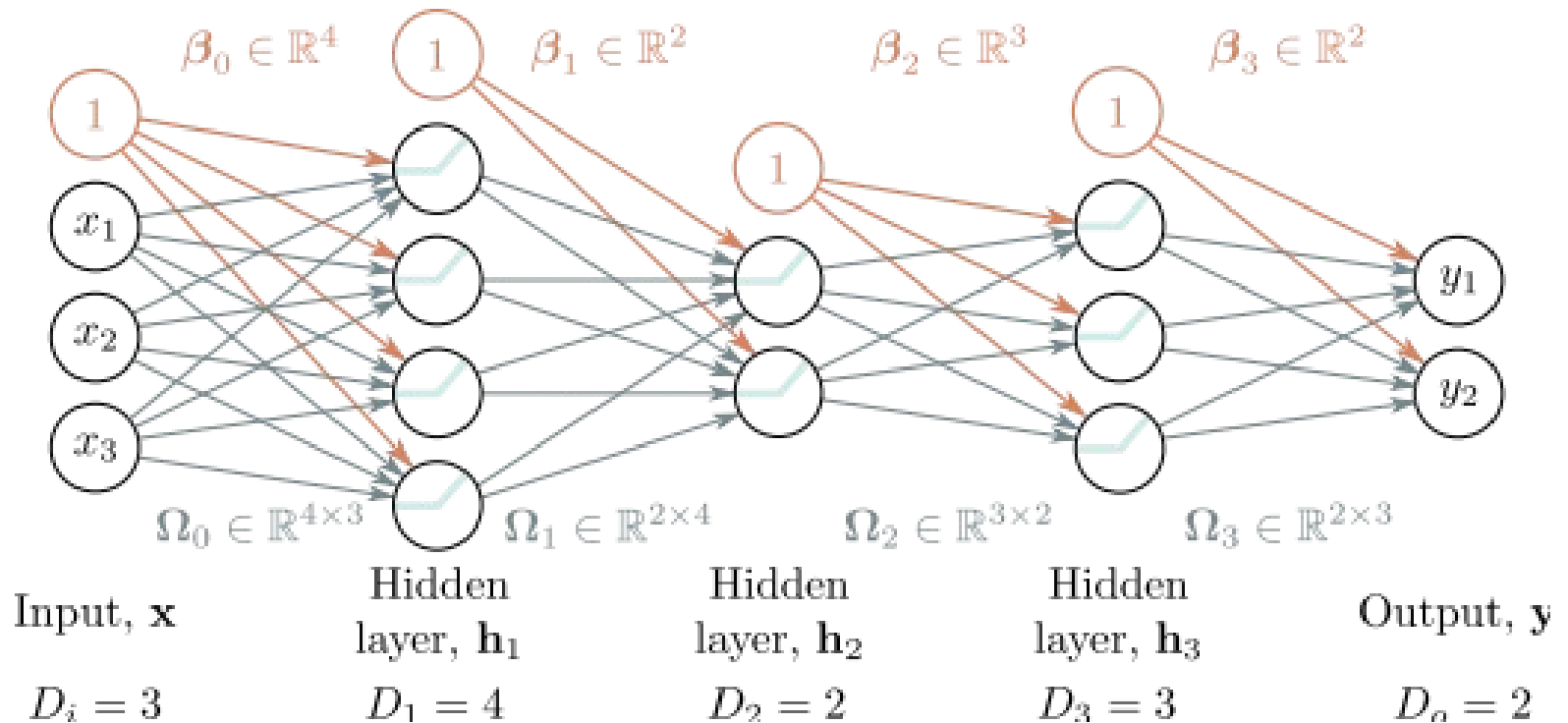


Activation Functions

- Let's have a look at possible activation functions.



Example: Network with 3 hidden layers



$$\mathbf{h}_1 = \mathbf{a}[\beta_0 + \Omega_0 \mathbf{x}]$$

$$\mathbf{h}_2 = \mathbf{a}[\beta_1 + \Omega_1 \mathbf{h}_1]$$

$$\mathbf{h}_3 = \mathbf{a}[\beta_2 + \Omega_2 \mathbf{h}_2]$$

$$\mathbf{f}[\mathbf{x}, \phi] = \beta_3 + \Omega_3 \mathbf{h}_3$$

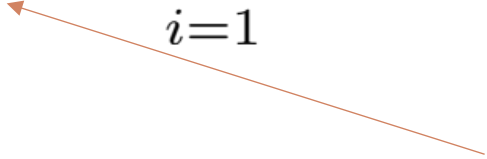
Loss Function

- Training dataset of I pairs of input/output examples:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^I$$

- **Loss function** or **cost function** measures how bad model is:

$$L[\phi] = \sum_{i=1}^I \ell_i = \sum_{i=1}^I l[f[\mathbf{x}_i, \phi], y_i]$$



Returns a scalar that is smaller when model maps inputs to outputs better

Problem 1:

Computing Gradients

Loss measures how bad model is:

$$L[\phi] = \sum_{i=1}^I \ell_i = \sum_{i=1}^I l[f[\mathbf{x}_i, \phi], y_i]$$

Stochastic Gradient Descent Algorithm:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

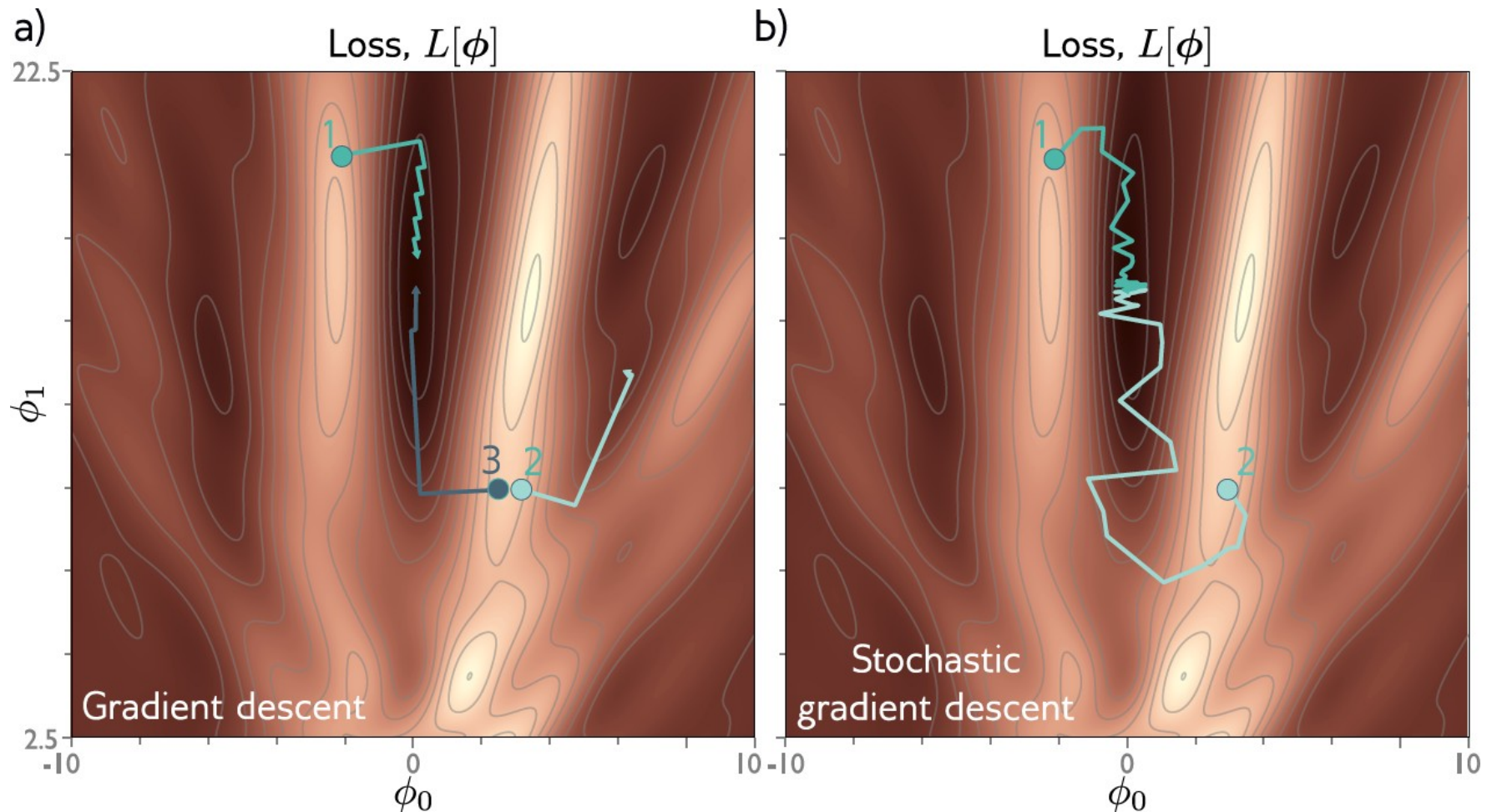
Parameters:

$$\phi = \{\beta_0, \Omega_0, \beta_1, \Omega_1, \beta_2, \Omega_2, \beta_3, \Omega_3\}$$

Need to compute gradients

$$\frac{\partial \ell_i}{\partial \beta_k} \quad \text{and} \quad \frac{\partial \ell_i}{\partial \Omega_k}$$

Problem 2: Initialization



Where should we start the parameters before we commence SGD?

Computing Gradients

- **Observation 1:**
Small change to the weight ω (element of $\mathbf{\Omega}_k$) is amplified or attenuated by the activation $\mathbf{h}$
- **Observation 2:**
A small change in a bias or weight causes a ripple of changes through the subsequent network.

Algorithm to compute gradient efficiently

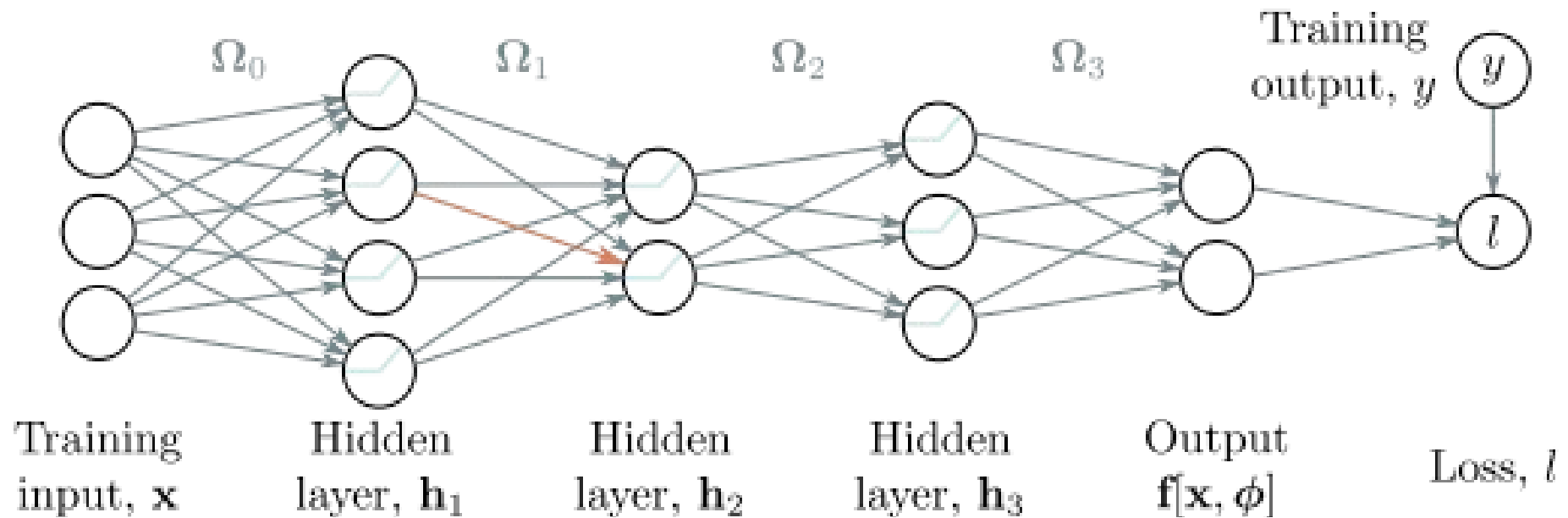
- “Backpropagation algorithm”
- Rumelhart, Hinton, and Williams (1986)

Gradients

- Backpropagation intuition
- Toy model
- Background mathematics
- Backpropagation forward pass
- Backpropagation backward pass
- Algorithmic differentiation
- Code

BackProp Intuition #1:

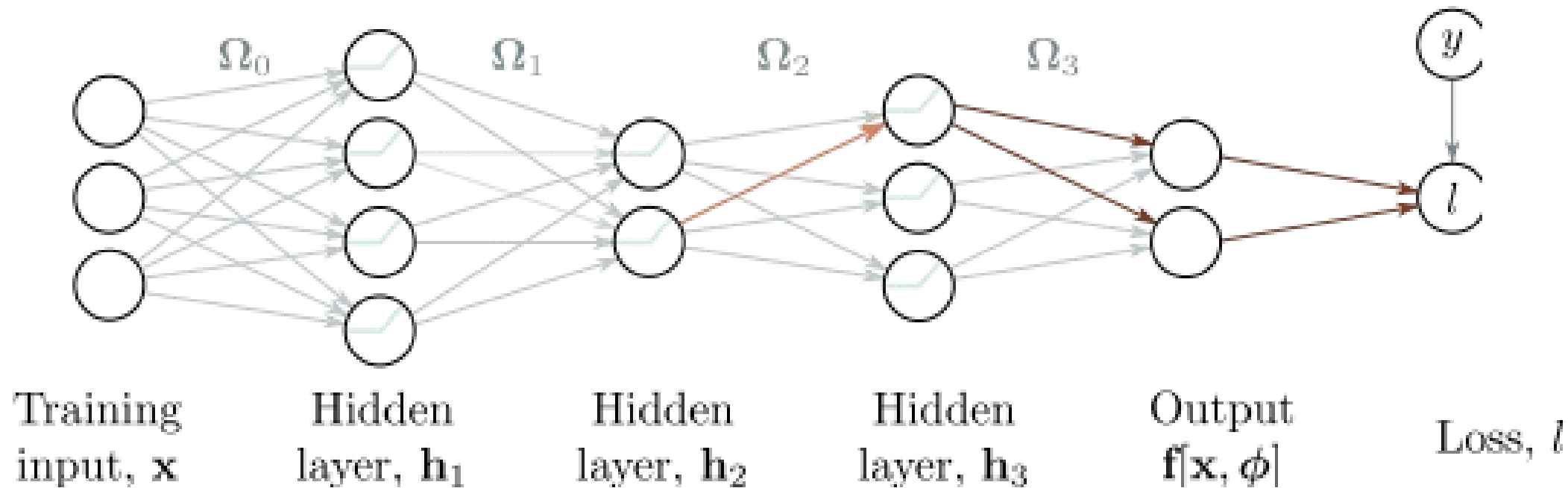
The Forward Pass



- Orange weight multiplies activation (ReLU output) from previous layer
- We want to know, how change in orange weight affects loss.
- If we double activation in previous layer, weight will have twice the effect.
- Conclusion: **we need to know the activations at each layer.**

BackProp Intuition #2:

The Backward Pass

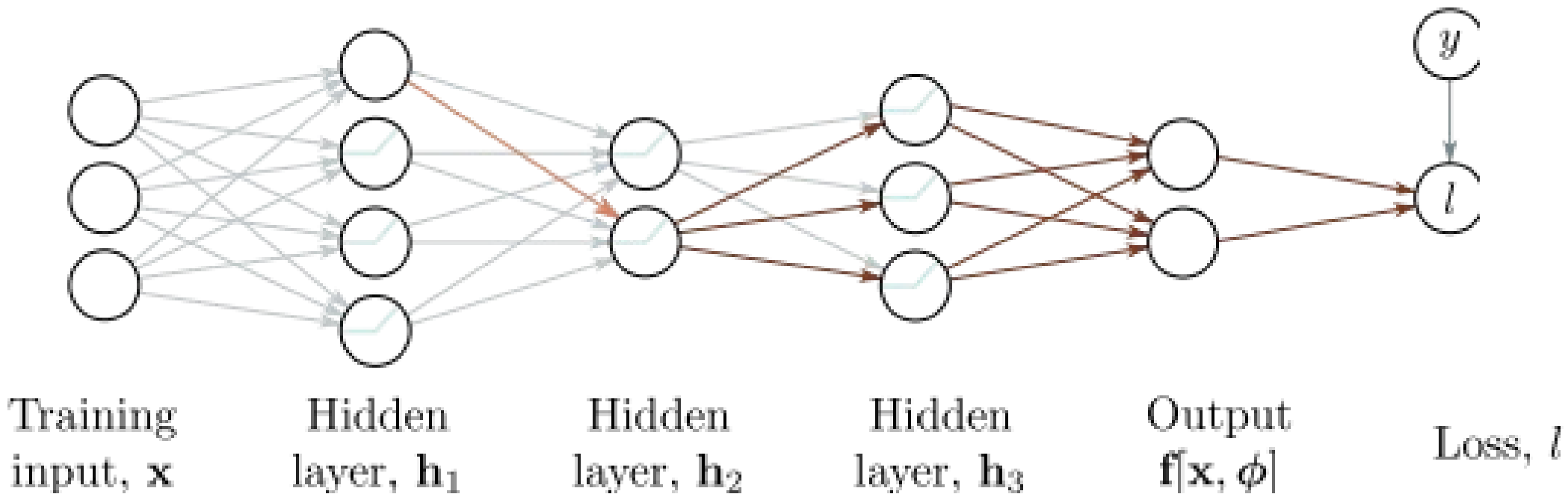


To calculate how a small change in a weight or bias feeding into hidden layer $\mathbf{h}_3$ modifies the loss, we need to know:

- how a change in layer $\mathbf{h}_3$ changes the model output $\mathbf{f}$
- how a change in model output changes the loss l

BackProp Intuition #2:

The Backward Pass

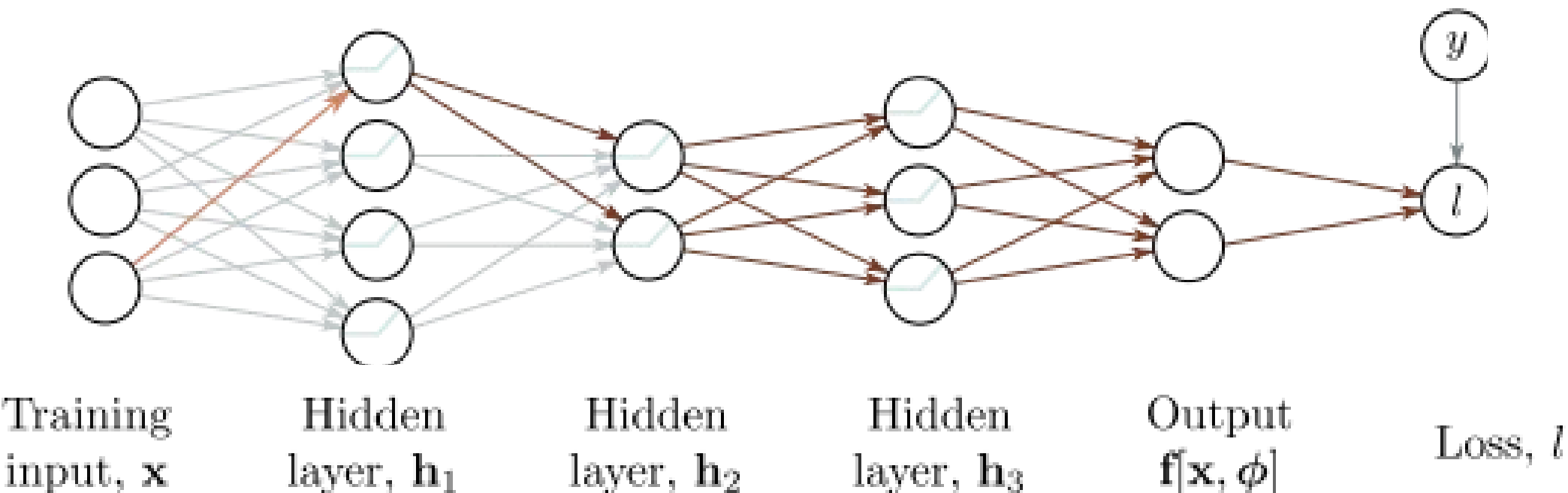


To calculate **how a small change in a weight or bias** feeding into hidden layer $\mathbf{h}_2$ **modifies the loss**, we need to know:

- how a change in layer $\mathbf{h}_2$ affects $\mathbf{h}_3$
- how $\mathbf{h}_3$ changes the model output
- how this output changes the loss

BackProp Intuition #2:

The Backward Pass



To calculate how a small change in a weight or bias feeding into hidden layer $\mathbf{h}_1$ modifies the loss, we need to know:

- how a change in layer $\mathbf{h}_1$ affects layer $\mathbf{h}_2$
- how a change in layer $\mathbf{h}_2$ affects layer $\mathbf{h}_3$
- how layer $\mathbf{h}_3$ changes the model output
- how the model output changes the loss

Gradients

- Backpropagation intuition
- Toy model
- Background mathematics
- Backpropagation forward pass
- Backpropagation backward pass
- Algorithmic differentiation
- Code

Toy function

$$f[x, \phi] = \beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x] \right] \right]$$

$$\ell_i = (f[x_i, \phi] - y_i)^2$$

- Consists of a series of functions that are composed with each other.
- Unlike in neural networks just uses scalars (not vectors)
- “Activation functions” sin, exp, cos

Toy function

$$f[x, \phi] = \beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x] \right] \right]$$

$$\ell_i = (f[x_i, \phi] - y_i)^2$$

Derivatives

$$\frac{\partial \cos[z]}{\partial z} = -\sin[z] \quad \frac{\partial \exp[z]}{\partial z} = \exp[z] \quad \frac{\partial \sin[z]}{\partial z} = \cos[z]$$

Gradients of toy function

$$f[x, \phi] = \beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x] \right] \right]$$

$$\ell_i = (f[x_i, \phi] - y_i)^2$$

We want to calculate:

How does a small change in β_3 change the loss ℓ_i for the i 'th example?

$$\frac{\partial \ell_i}{\partial \beta_0}, \quad \frac{\partial \ell_i}{\partial \omega_0}, \quad \frac{\partial \ell_i}{\partial \beta_1}, \quad \frac{\partial \ell_i}{\partial \omega_1}, \quad \frac{\partial \ell_i}{\partial \beta_2}, \quad \frac{\partial \ell_i}{\partial \omega_2}, \quad \frac{\partial \ell_i}{\partial \beta_3}, \quad \text{and} \quad \frac{\partial \ell_i}{\partial \omega_3}$$

Gradients of Composed Functions

$$f[x, \phi] = \beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x] \right] \right]$$

$$\ell_i = (f[x_i, \phi] - y_i)^2$$

Calculating expressions by hand:

- some expressions very complicated.
- obvious redundancy (look at sin terms in bottom equation)

$$\begin{aligned} \frac{\partial \ell_i}{\partial \omega_0} = & -2 \left(\beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x_i] \right] \right] - y_i \right) \\ & \cdot \omega_1 \omega_2 \omega_3 \cdot x_i \cdot \cos [\beta_0 + \omega_0 \cdot x_i] \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x_i] \right] \\ & \cdot \sin \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x_i] \right] \right] \end{aligned}$$

Forward Pass

$$f[x, \phi] = \beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x] \right] \right]$$

$$\ell_i = (f[x_i, \phi] - y_i)^2$$

1. Write this as a series of intermediate calculations
2. Compute these intermediate quantities

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

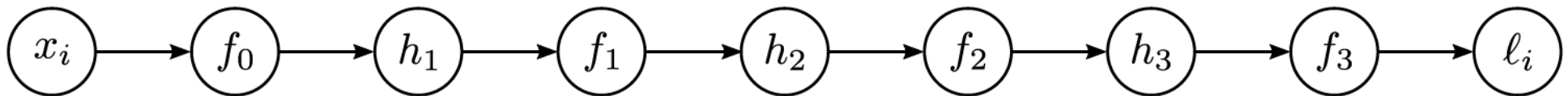
$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$



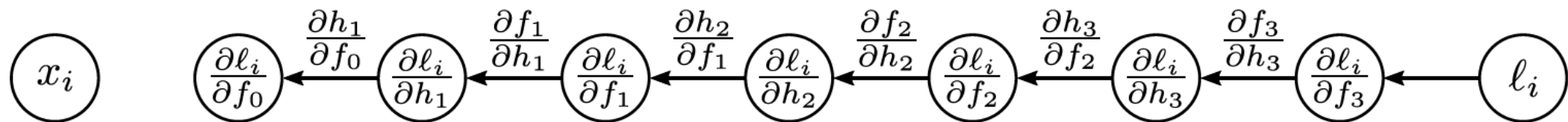
Backward Pass

$$f[x, \phi] = \beta_3 + \omega_3 \cdot \cos \left[\beta_2 + \omega_2 \cdot \exp \left[\beta_1 + \omega_1 \cdot \sin [\beta_0 + \omega_0 \cdot x] \right] \right]$$

$$\ell_i = (f[x_i, \phi] - y_i)^2$$

1. Compute the derivatives of the loss with respect to these intermediate quantities, but in reverse order.

$$\frac{\partial \ell_i}{\partial f_3}, \quad \frac{\partial \ell_i}{\partial h_3}, \quad \frac{\partial \ell_i}{\partial f_2}, \quad \frac{\partial \ell_i}{\partial h_2}, \quad \frac{\partial \ell_i}{\partial f_1}, \quad \frac{\partial \ell_i}{\partial h_1}, \quad \text{and} \quad \frac{\partial \ell_i}{\partial f_0}$$



Backward Pass

1. Compute the derivatives of the loss with respect to these intermediate quantities, but in reverse order.

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- The first of these derivatives is trivial

$$\frac{\partial \ell_i}{\partial f_3} = 2(f_3 - y_i)$$

Backward Pass

1. Compute the derivatives of the loss with respect to these intermediate quantities, but in reverse order.

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- The second of these derivatives is computed via the chain rule

$$\frac{\partial \ell_i}{\partial h_3} = \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3}$$

How does a small change in h_3 change ℓ_i ?

How does a small change in h_3 change f_3 ?

How does a small change in f_3 change ℓ_i ?

Backward Pass

1. Compute the derivatives of the loss with respect to these intermediate quantities, but in reverse order.

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- The second of these derivatives is computed via the chain rule

$$\frac{\partial \ell_i}{\partial h_3} = \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3}$$

How does a small change in h_3 change ℓ_i ?

ω_3

Already computed!

Backward Pass

1. Compute the derivatives of the loss with respect to these intermediate quantities, but in reverse order.

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- The remaining derivatives are also calculated by further use of chain rule

$$\frac{\partial \ell_i}{\partial f_2} = \frac{\partial h_3}{\partial f_2} \left(\frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$-\sin[f_2]$

Already
computed!

Backward Pass

1. Compute the derivatives of the loss with respect to these intermediate quantities, but in reverse order.

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- The remaining derivatives are also calculated by further use of chain rule

$$\frac{\partial \ell_i}{\partial f_2} = \frac{\partial h_3}{\partial f_2} \left(\frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$$\frac{\partial \ell_i}{\partial h_2} = \frac{\partial f_2}{\partial h_2} \left(\frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$$\frac{\partial \ell_i}{\partial f_1} = \frac{\partial h_2}{\partial f_1} \left(\frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$$\frac{\partial \ell_i}{\partial h_1} = \frac{\partial f_1}{\partial h_1} \left(\frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$$\frac{\partial \ell_i}{\partial f_0} = \frac{\partial h_1}{\partial f_0} \left(\frac{\partial f_1}{\partial h_1} \frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

Backward Pass

1. Compute the derivatives of the loss with respect to these intermediate quantities, but in reverse order.

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

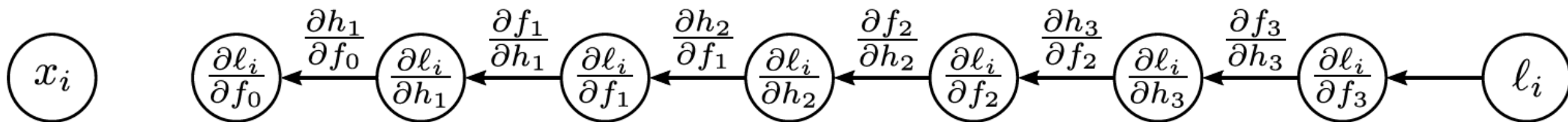
$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- The remaining derivatives are also calculated by further use of chain rule.



Backward Pass

2. Find how the loss changes as a function of the parameters β and ω .

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- Another application of the chain rule

$$\frac{\partial \ell_i}{\partial \omega_k} = \frac{\partial f_k}{\partial \omega_k} \frac{\partial \ell_i}{\partial f_k}$$

How does a small change in ω_k change ℓ_i ?

How does a small change in ω_k change f_k ?

How does a small change in f_k change ℓ_i ?

Backward Pass

2. Find how the loss changes as a function of the parameters β and ω .

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- Another application of the chain rule

$$\frac{\partial \ell_i}{\partial \omega_k} = \frac{\partial f_k}{\partial \omega_k} \frac{\partial \ell_i}{\partial f_k}$$

How does a small change in change ?

h_k

Already calculated in part 1.

Backward Pass

2. Find how the loss changes as a function of the parameters β and ω .

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$

- Another application of the chain rule
- Similarly for β parameters

$$\frac{\partial \ell_i}{\partial \omega_k} = \frac{\partial f_k}{\partial \omega_k} \frac{\partial \ell_i}{\partial f_k}$$

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial f_k}{\partial \beta_k} \frac{\partial \ell_i}{\partial f_k}$$

Backward Pass

2. Find how the loss changes as a function of the parameters β and ω .

$$f_0 = \beta_0 + \omega_0 \cdot x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 \cdot h_1$$

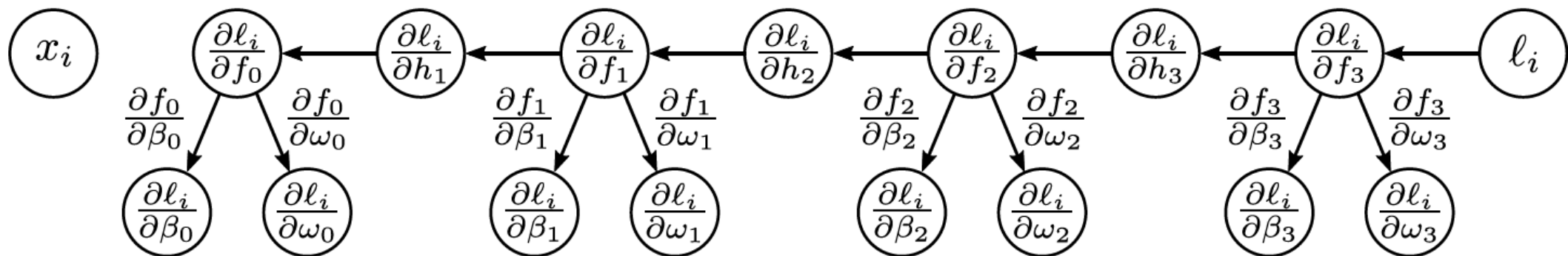
$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + \omega_2 \cdot h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 \cdot h_3$$

$$\ell_i = (f_3 - y_i)^2.$$



Gradients

- Backpropagation intuition
- Toy model
- Background mathematics
- Backpropagation forward pass
- Backpropagation backward pass
- Algorithmic differentiation
- Code

Matrix Calculus

Scalar function $f[\cdot]$ of
a vector $\mathbf{a}$

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix}$$

$$\frac{\partial f}{\partial \mathbf{a}} = \begin{bmatrix} \frac{\partial f}{\partial a_1} \\ \frac{\partial f}{\partial a_2} \\ \frac{\partial f}{\partial a_3} \\ \frac{\partial f}{\partial a_4} \end{bmatrix}$$

Matrix Calculus

Scalar function $f[\cdot]$ of a matrix $\mathbf{A}$

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{bmatrix} \qquad \frac{\partial f}{\partial \mathbf{A}} = \begin{bmatrix} \frac{\partial f}{\partial a_{11}} & \frac{\partial f}{\partial a_{12}} & \frac{\partial f}{\partial a_{13}} \\ \frac{\partial f}{\partial a_{21}} & \frac{\partial f}{\partial a_{22}} & \frac{\partial f}{\partial a_{23}} \\ \frac{\partial f}{\partial a_{31}} & \frac{\partial f}{\partial a_{32}} & \frac{\partial f}{\partial a_{33}} \\ \frac{\partial f}{\partial a_{41}} & \frac{\partial f}{\partial a_{42}} & \frac{\partial f}{\partial a_{43}} \end{bmatrix}$$

Matrix Calculus

Vector function $\mathbf{f}[\cdot]$ of vector $\mathbf{a}$

$$\mathbf{f} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix} \quad \mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} \quad \frac{\partial \mathbf{f}}{\partial \mathbf{a}} = \begin{bmatrix} \frac{\partial f_1}{\partial a_1} & \frac{\partial f_2}{\partial a_1} & \frac{\partial f_3}{\partial a_1} \\ \frac{\partial f_1}{\partial a_2} & \frac{\partial f_2}{\partial a_2} & \frac{\partial f_3}{\partial a_2} \\ \frac{\partial f_1}{\partial a_3} & \frac{\partial f_2}{\partial a_3} & \frac{\partial f_3}{\partial a_3} \\ \frac{\partial f_1}{\partial a_4} & \frac{\partial f_2}{\partial a_4} & \frac{\partial f_3}{\partial a_4} \end{bmatrix}$$

Comparing Vector and Matrix

Scalar derivatives:

$$f_3 = \beta_3 + \omega_3 h_3 \quad \frac{\partial f_3}{\partial h_3} = \frac{\partial}{\partial h_3} (\beta_3 + \omega_3 h_3) = \omega_3$$

Matrix derivatives:

$$\mathbf{f}_3 = \boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3 \quad \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} = \frac{\partial}{\partial \mathbf{h}_3} (\boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3) = \boldsymbol{\Omega}_3^T$$

Comparing Vector and Matrix

Scalar derivatives:

$$f_3 = \beta_3 + \omega_3 h_3 \qquad \frac{\partial f_3}{\partial \beta_3} = \frac{\partial}{\partial \omega_3} \beta_3 + \omega_3 h_3 = 1$$

Matrix derivatives:

$$\mathbf{f}_3 = \boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3 \qquad \frac{\partial \mathbf{f}_3}{\partial \boldsymbol{\beta}_3} = \frac{\partial}{\partial \boldsymbol{\beta}_3} (\boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3) = \mathbf{I}$$

Homework: (keeners only)

- Consider function: $\mathbf{f} = \mathbf{B}\mathbf{a}$ $\mathbf{f} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix}$ $\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix}$

- Can write as: $f_i = \sum_j B_{ij} a_j$

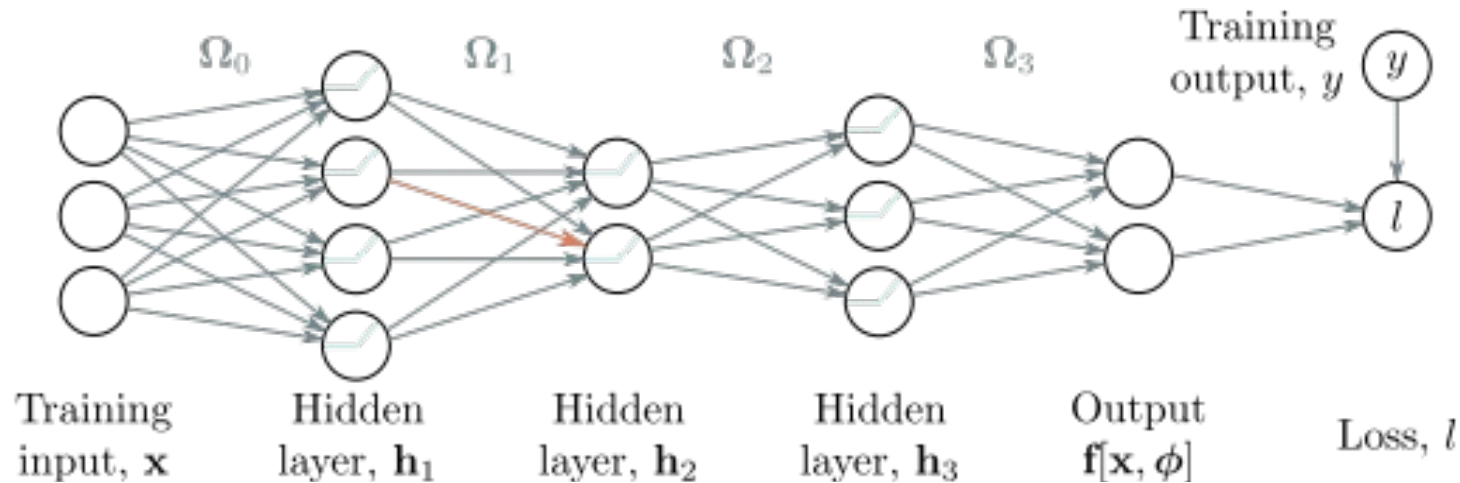
- Now calculate: $\frac{\partial \mathbf{f}}{\partial \mathbf{a}} = \begin{bmatrix} \frac{\partial f_1}{\partial a_1} & \frac{\partial f_2}{\partial a_1} & \frac{\partial f_3}{\partial a_1} \\ \frac{\partial f_1}{\partial a_2} & \frac{\partial f_2}{\partial a_2} & \frac{\partial f_3}{\partial a_2} \\ \frac{\partial f_1}{\partial a_3} & \frac{\partial f_2}{\partial a_3} & \frac{\partial f_3}{\partial a_3} \\ \frac{\partial f_1}{\partial a_4} & \frac{\partial f_2}{\partial a_4} & \frac{\partial f_3}{\partial a_4} \end{bmatrix}$

- Write final expression as a matrix

Gradients

- Backpropagation intuition
- Toy model
- Background mathematics
- Backpropagation forward pass
- Backpropagation backward pass
- Algorithmic differentiation
- Code

The Forward Pass



1. Compute a series of intermediate quantities

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

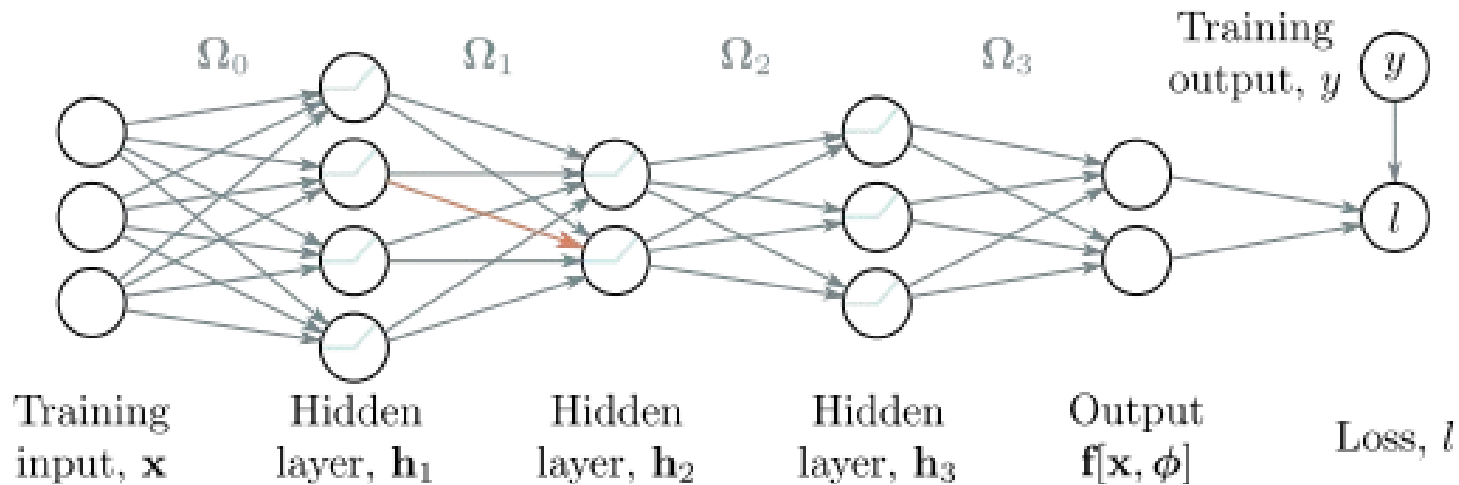
$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

Gradients

- Backpropagation intuition
- Toy model
- Background mathematics
- Backpropagation forward pass
- Backpropagation backward pass
- Algorithmic differentiation
- Code

The Backward Pass



1. Compute a series of intermediate quantities

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

Yikes!

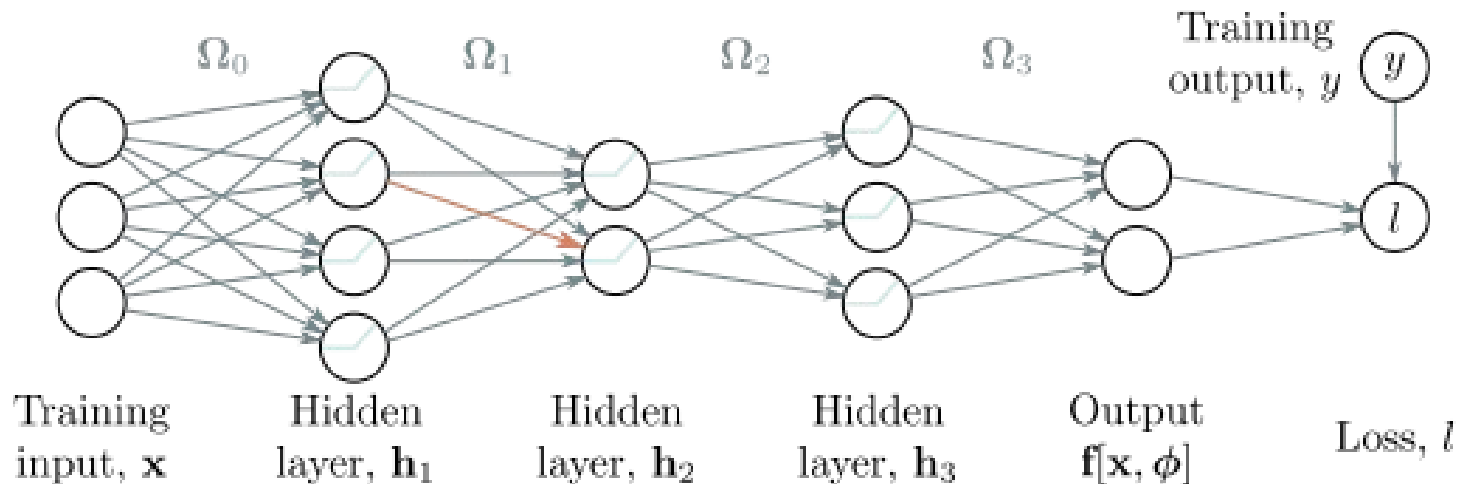
- But:

$$\frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} = \frac{\partial}{\partial \mathbf{h}_3} (\boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3) = \boldsymbol{\Omega}_3^T$$

- Quite similar to:

$$\frac{\partial f_3}{\partial h_3} = \frac{\partial}{\partial h_3} (\beta_3 + \omega_3 h_3) = \omega_3$$

The Backward Pass



1. Compute a series of intermediate quantities

$$\begin{aligned} \mathbf{f}_0 &= \beta_0 + \Omega_0 \mathbf{x}_i \\ \mathbf{h}_1 &= \mathbf{a}[\mathbf{f}_0] \\ \mathbf{f}_1 &= \beta_1 + \Omega_1 \mathbf{h}_1 \\ \mathbf{h}_2 &= \mathbf{a}[\mathbf{f}_1] \\ \mathbf{f}_2 &= \beta_2 + \Omega_2 \mathbf{h}_2 \\ \mathbf{h}_3 &= \mathbf{a}[\mathbf{f}_2] \\ \mathbf{f}_3 &= \beta_3 + \Omega_3 \mathbf{h}_3 \\ \ell_i &= l[\mathbf{f}_3, y_i] \end{aligned}$$

2. Take derivatives of output with respect to intermediate quantities

$$\frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

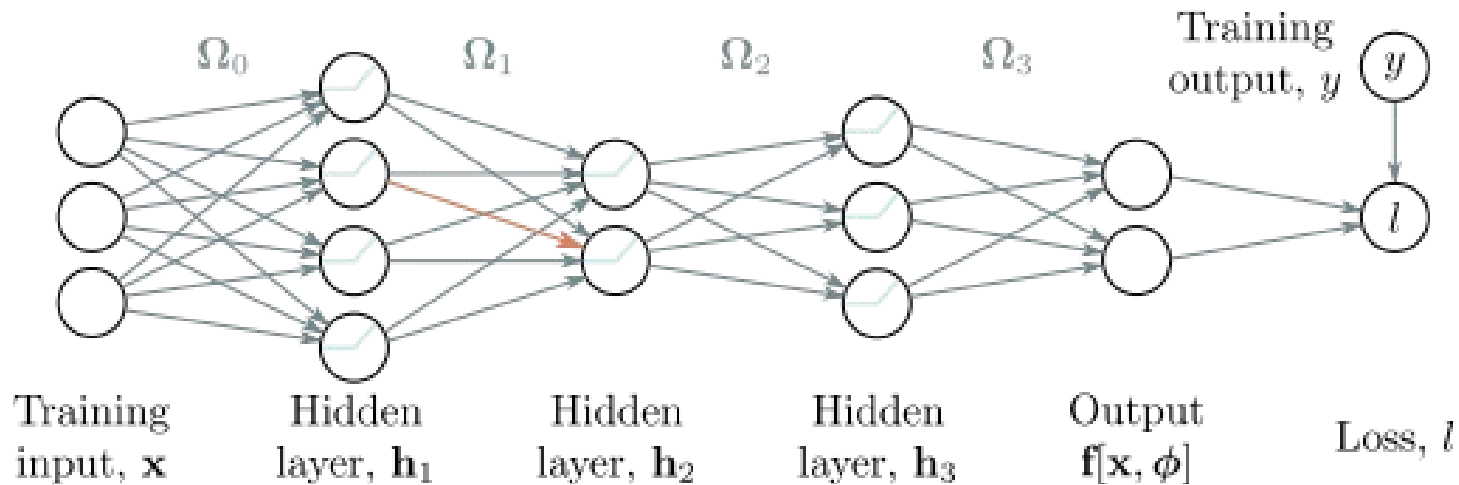
$$\frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} = \frac{\partial}{\partial \mathbf{h}_3} (\beta_3 + \Omega_3 \mathbf{h}_3) = \Omega_3^T$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

The Backward Pass



1. Compute a series of intermediate quantities

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

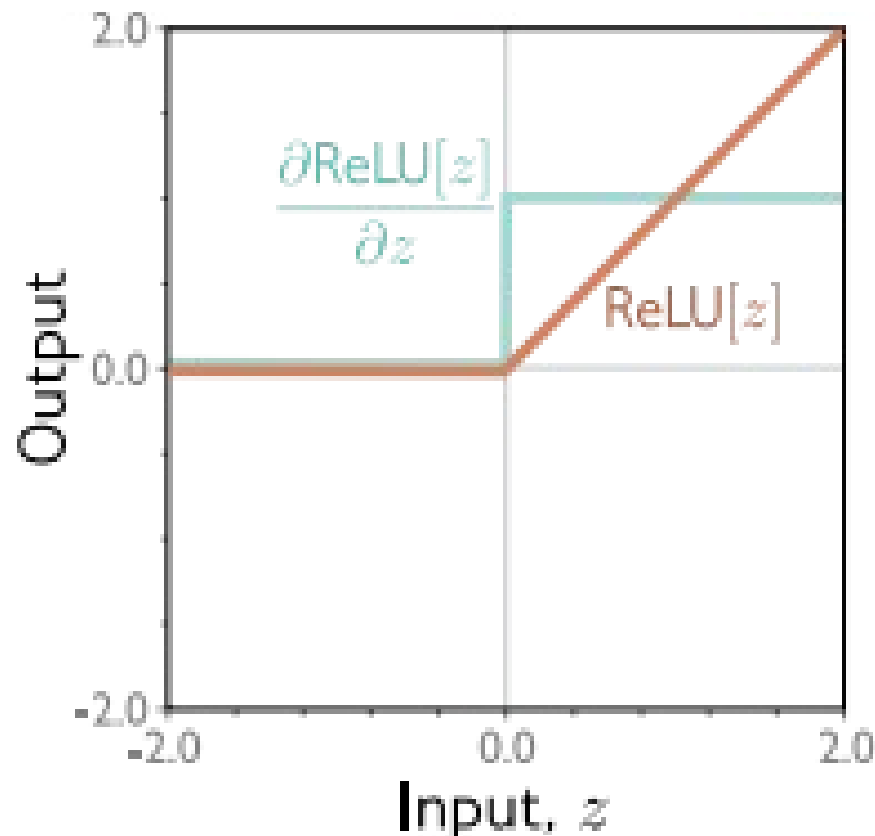
$$\frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

Derivative of ReLU



$\mathbb{I}[z > 0]$
"Indicator
function"

Derivative of ReLU

1. Consider:

$$\mathbf{a} = \text{ReLU}[\mathbf{b}]$$

where:

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Derivative of ReLU

2. We could equivalently write:

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \text{ReLU}[b_1] \\ \text{ReLU}[b_2] \\ \text{ReLU}[b_3] \end{bmatrix}$$

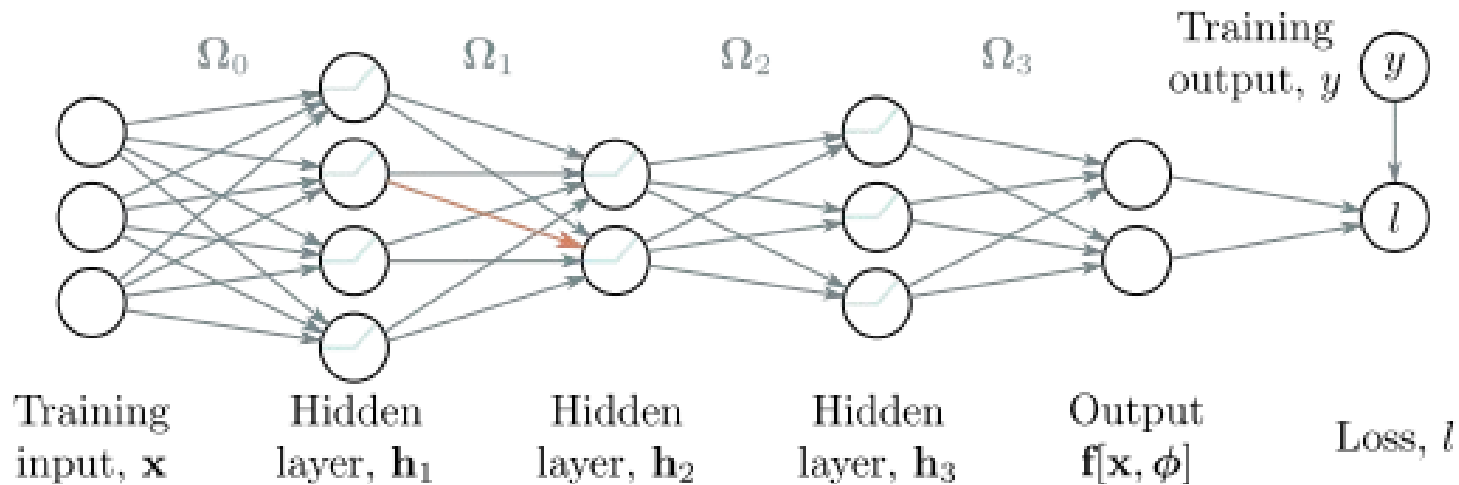
3. Taking the derivative

$$\frac{\partial \mathbf{a}}{\partial \mathbf{b}} = \begin{bmatrix} \frac{\partial a_1}{\partial b_1} & \frac{\partial a_2}{\partial b_1} & \frac{\partial a_3}{\partial b_1} \\ \frac{\partial a_1}{\partial b_2} & \frac{\partial a_2}{\partial b_2} & \frac{\partial a_3}{\partial b_2} \\ \frac{\partial a_1}{\partial b_3} & \frac{\partial a_2}{\partial b_3} & \frac{\partial a_3}{\partial b_3} \end{bmatrix} = \begin{bmatrix} \mathbb{I}[b_1 > 0] & 0 & 0 \\ 0 & \mathbb{I}[b_2 > 0] & 0 \\ 0 & 0 & \mathbb{I}[b_3 > 0] \end{bmatrix}$$

4. We can equivalently pointwise multiply by diagonal

$$\mathbb{I}[\mathbf{b} > 0] \odot$$

The Backward Pass



1. Compute a series of intermediate quantities

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$$

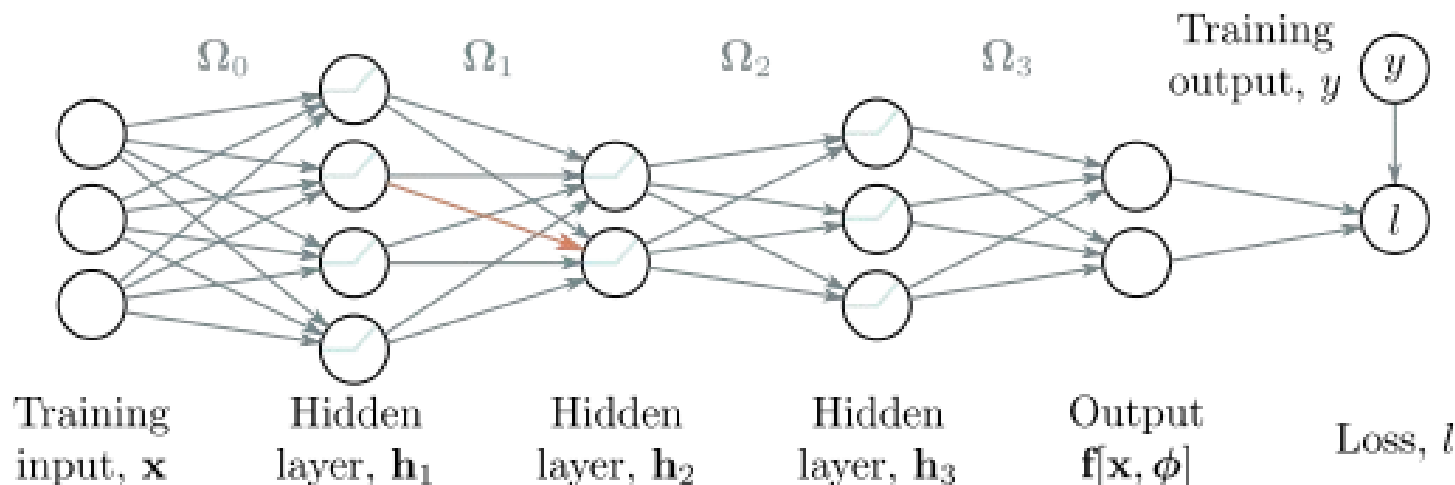
$$\frac{\partial \ell_i}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \left(\frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_0} = \frac{\partial \mathbf{h}_1}{\partial \mathbf{f}_0} \frac{\partial \mathbf{f}_1}{\partial \mathbf{h}_1} \left(\frac{\partial \mathbf{h}_2}{\partial \mathbf{f}_1} \frac{\partial \mathbf{f}_2}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3} \right)$$

$$\mathbb{I}[\mathbf{f}_2 > 0]$$

2. Take derivatives of output with respect to intermediate quantities

The Backward Pass



1. Compute a series of intermediate quantities

$$\mathbf{f}_0 = \beta_0 + \Omega_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \beta_1 + \Omega_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

$$\mathbf{f}_2 = \beta_2 + \Omega_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \beta_3 + \Omega_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

2. Take derivatives of output with respect to intermediate Quantities

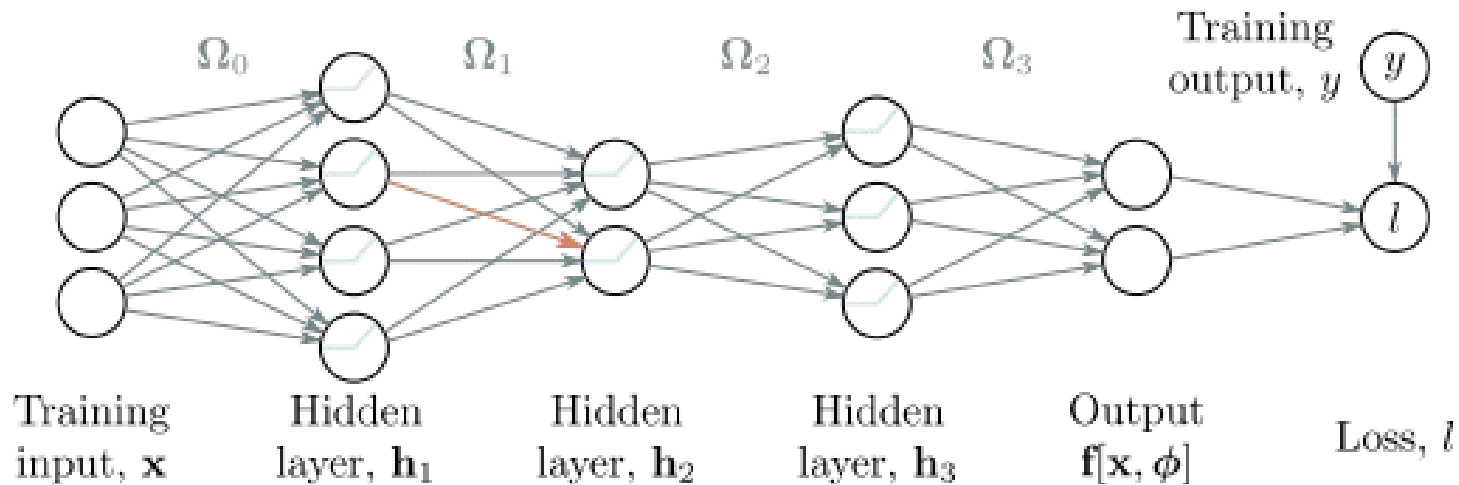
3. Take derivatives w.r.t. bias parameters β

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial \mathbf{f}_k}{\partial \beta_k} \frac{\partial \ell_i}{\partial \mathbf{f}_k}$$

$$= \frac{\partial}{\partial \beta_k} (\beta_k + \Omega_k \mathbf{h}_k) \frac{\partial \ell_i}{\partial \mathbf{f}_k}$$

$$= \frac{\partial \ell_i}{\partial \mathbf{f}_k},$$

The Backward Pass



1. Compute a series of intermediate quantities

$$\begin{aligned} \mathbf{f}_0 &= \beta_0 + \Omega_0 \mathbf{x}_i \\ \mathbf{h}_1 &= \mathbf{a}[\mathbf{f}_0] \\ \mathbf{f}_1 &= \beta_1 + \Omega_1 \mathbf{h}_1 \\ \mathbf{h}_2 &= \mathbf{a}[\mathbf{f}_1] \\ \mathbf{f}_2 &= \beta_2 + \Omega_2 \mathbf{h}_2 \\ \mathbf{h}_3 &= \mathbf{a}[\mathbf{f}_2] \\ \mathbf{f}_3 &= \beta_3 + \Omega_3 \mathbf{h}_3 \\ \ell_i &= l[\mathbf{f}_3, y_i] \end{aligned}$$

2. Take derivatives of output with respect to intermediate Quantities

3. Take derivatives w.r.t. weight parameters Ω

$$\begin{aligned} \frac{\partial \ell_i}{\partial \Omega_k} &= \frac{\partial \mathbf{f}_k}{\partial \Omega_k} \frac{\partial \ell_i}{\partial \mathbf{f}_k} \\ &= \frac{\partial}{\partial \Omega_k} (\beta_k + \Omega_k \mathbf{h}_k) \frac{\partial \ell_i}{\partial \mathbf{f}_k} \\ &= \frac{\partial \ell_i}{\partial \mathbf{f}_k} \mathbf{h}_k^T \end{aligned}$$

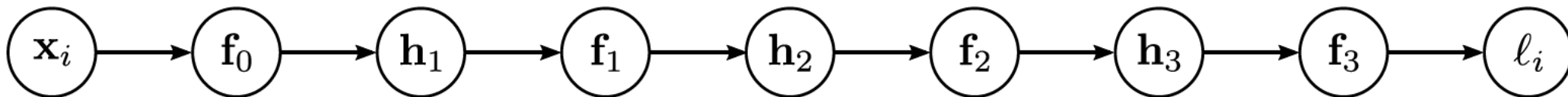
Backprop Summary

Forward pass: We compute and store the following quantities:

$$\mathbf{f}_0 = \boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}_i$$

$$\mathbf{h}_k = \mathbf{a}[\mathbf{f}_{k-1}] \quad k \in \{1, 2, \dots, K\}$$

$$\mathbf{f}_k = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{h}_k. \quad k \in \{1, 2, \dots, K\}$$



Backprop Summary

Backward pass: We start with the derivative $\partial\ell_i/\partial\mathbf{f}_K$ of the loss function ℓ_i with respect to the network output $\mathbf{f}_K$ and work backward through the network:

$$\begin{aligned}\frac{\partial\ell_i}{\partial\boldsymbol{\beta}_k} &= \frac{\partial\ell_i}{\partial\mathbf{f}_k} & k \in \{K, K-1, \dots, 1\} \\ \frac{\partial\ell_i}{\partial\boldsymbol{\Omega}_k} &= \frac{\partial\ell_i}{\partial\mathbf{f}_k} \mathbf{h}_k^T & k \in \{K, K-1, \dots, 1\} \\ \frac{\partial\ell_i}{\partial\mathbf{f}_{k-1}} &= \mathbb{I}[\mathbf{f}_{k-1} > 0] \odot \left(\boldsymbol{\Omega}_k^T \frac{\partial\ell_i}{\partial\mathbf{f}_k} \right) & k \in \{K, K-1, \dots, 1\}\end{aligned}$$

where $\odot$ denotes pointwise multiplication and $\mathbb{I}[\mathbf{f}_{k-1} > 0]$ is a vector containing ones where $\mathbf{f}_{k-1}$ is greater than zero and zeros elsewhere. Finally, we compute the derivatives with respect to the first set of biases and weights:

Backprop Summary

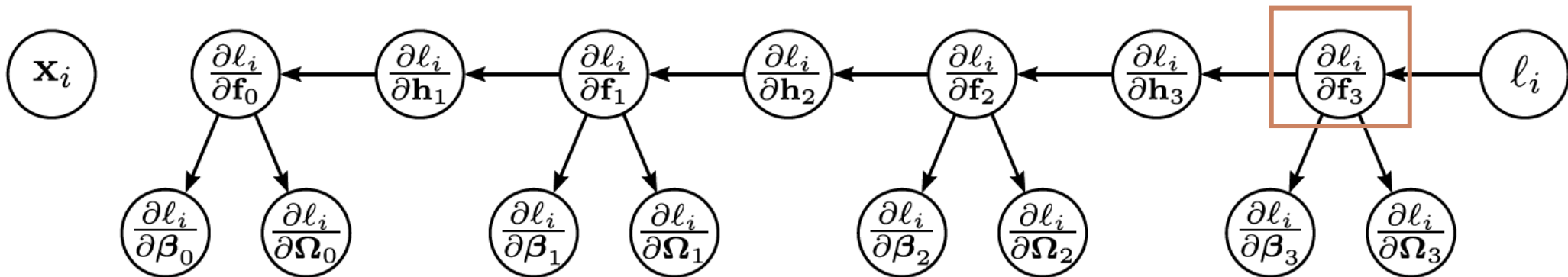
Backward pass: We start with the derivative $\frac{\partial \ell_i}{\partial \mathbf{f}_K}$ of the loss function ℓ_i with respect to the network output $\mathbf{f}_K$ and work backward through the network:

$$\frac{\partial \ell_i}{\partial \boldsymbol{\beta}_k} = \frac{\partial \ell_i}{\partial \mathbf{f}_k} \quad k \in \{K, K-1, \dots, 1\}$$

$$\frac{\partial \ell_i}{\partial \boldsymbol{\Omega}_k} = \frac{\partial \ell_i}{\partial \mathbf{f}_k} \mathbf{h}_k^T \quad k \in \{K, K-1, \dots, 1\}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_{k-1}} = \mathbb{I}[\mathbf{f}_{k-1} > 0] \odot \left(\boldsymbol{\Omega}_k^T \frac{\partial \ell_i}{\partial \mathbf{f}_k} \right) \quad k \in \{K, K-1, \dots, 1\}$$

where $\odot$ denotes pointwise multiplication and $\mathbb{I}[\mathbf{f}_{k-1} > 0]$ is a vector containing ones where $\mathbf{f}_{k-1}$ is greater than zero and zeros elsewhere. Finally, we compute the derivatives with respect to the first set of biases and weights:



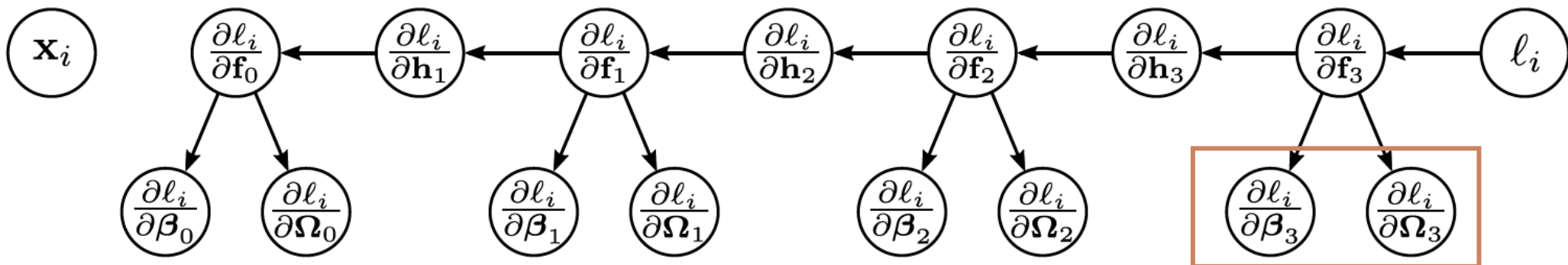
Backprop Summary

Backward pass: We start with the derivative $\partial \ell_i / \partial \mathbf{f}_K$ of the loss function ℓ_i with respect to the network output $\mathbf{f}_K$ and work backward through the network:

$$\boxed{\begin{aligned} \frac{\partial \ell_i}{\partial \beta_k} &= \frac{\partial \ell_i}{\partial \mathbf{f}_k} \\ \frac{\partial \ell_i}{\partial \Omega_k} &= \frac{\partial \ell_i}{\partial \mathbf{f}_k} \mathbf{h}_k^T \end{aligned}} \quad \begin{aligned} k &\in \{K, K-1, \dots, 1\} \\ k &\in \{K, K-1, \dots, 1\} \end{aligned}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_{k-1}} = \mathbb{I}[\mathbf{f}_{k-1} > 0] \odot \left(\Omega_k^T \frac{\partial \ell_i}{\partial \mathbf{f}_k} \right) \quad k \in \{K, K-1, \dots, 1\}$$

where $\odot$ denotes pointwise multiplication and $\mathbb{I}[\mathbf{f}_{k-1} > 0]$ is a vector containing ones where $\mathbf{f}_{k-1}$ is greater than zero and zeros elsewhere. Finally, we compute the derivatives with respect to the first set of biases and weights:



Backprop Summary

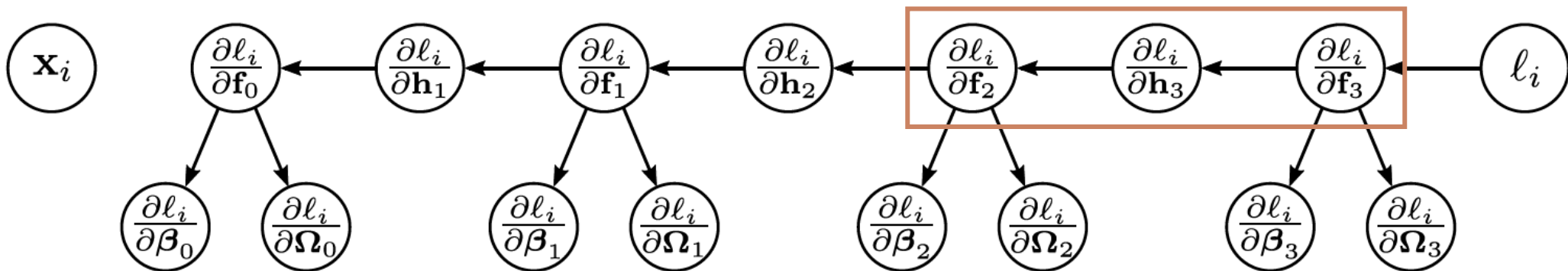
Backward pass: We start with the derivative $\partial \ell_i / \partial \mathbf{f}_K$ of the loss function ℓ_i with respect to the network output $\mathbf{f}_K$ and work backward through the network:

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial \ell_i}{\partial \mathbf{f}_k} \quad k \in \{K, K-1, \dots, 1\}$$

$$\frac{\partial \ell_i}{\partial \Omega_k} = \frac{\partial \ell_i}{\partial \mathbf{f}_k} \mathbf{h}_k^T \quad k \in \{K, K-1, \dots, 1\}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_{k-1}} = \mathbb{I}[\mathbf{f}_{k-1} > 0] \odot \left(\Omega_k^T \frac{\partial \ell_i}{\partial \mathbf{f}_k} \right) \quad k \in \{K, K-1, \dots, 1\}$$

where $\odot$ denotes pointwise multiplication and $\mathbb{I}[\mathbf{f}_{k-1} > 0]$ is a vector containing ones where $\mathbf{f}_{k-1}$ is greater than zero and zeros elsewhere. Finally, we compute the derivatives with respect to the first set of biases and weights:



Backprop Summary

Backward pass: We start with the derivative $\partial\ell_i/\partial\mathbf{f}_K$ of the loss function ℓ_i with respect to the network output $\mathbf{f}_K$ and work backward through the network:

$$\begin{aligned}\frac{\partial\ell_i}{\partial\boldsymbol{\beta}_k} &= \frac{\partial\ell_i}{\partial\mathbf{f}_k} & k \in \{K, K-1, \dots, 1\} \\ \frac{\partial\ell_i}{\partial\boldsymbol{\Omega}_k} &= \frac{\partial\ell_i}{\partial\mathbf{f}_k} \mathbf{h}_k^T & k \in \{K, K-1, \dots, 1\} \\ \frac{\partial\ell_i}{\partial\mathbf{f}_{k-1}} &= \mathbb{I}[\mathbf{f}_{k-1} > 0] \odot \left(\boldsymbol{\Omega}_k^T \frac{\partial\ell_i}{\partial\mathbf{f}_k} \right) & k \in \{K, K-1, \dots, 1\}\end{aligned}$$

where $\odot$ denotes pointwise multiplication and $\mathbb{I}[\mathbf{f}_{k-1} > 0]$ is a vector containing ones where $\mathbf{f}_{k-1}$ is greater than zero and zeros elsewhere. Finally, we compute the derivatives with respect to the first set of biases and weights:

$$\begin{aligned}\frac{\partial\ell_i}{\partial\boldsymbol{\beta}_0} &= \frac{\partial\ell_i}{\partial\mathbf{f}_0} \\ \frac{\partial\ell_i}{\partial\boldsymbol{\Omega}_0} &= \frac{\partial\ell_i}{\partial\mathbf{f}_0} \mathbf{x}_i^T\end{aligned}$$

Pros and Cons

- Extremely efficient
 - Only need **matrix multiplication** and **thresholding for ReLU** functions
- Memory hungry:
must store all the intermediate quantities
- Sequential
 - can process multiple batches in parallel
 - but things get harder,
if the whole model doesn't fit on one machine.

Gradients

- Backpropagation intuition
- Toy model
- Background mathematics
- Backpropagation forward pass
- Backpropagation backward pass
- **Algorithmic differentiation**
- Code

Algorithmic Differentiation

- Modern deep learning frameworks compute **derivatives automatically**.
- You just have to specify the **model** and the **loss**.
- How? **Algorithmic differentiation**
 - Each component knows how to compute its own derivative
 - **ReLU** knows how to compute derivative of its output w.r.t. **its input**
 - **Linear function** knows how to compute derivative of output w.r.t. **input**
 - **Linear function** knows how to compute derivative of output w.r.t. **parameter**
 - You specify the order of the components.
 - Algorithm can compute the chain of derivatives.
- Works with branches as long as it is still an **acyclic graph**.

Gradients

- Backpropagation intuition
- Toy model
- Background mathematics
- Backpropagation forward pass
- Backpropagation backward pass
- Algorithmic differentiation
- Code

PyTorch Code

- Define a neural network
- Initialize parameters
- Define loss function
- Choose optimization algorithm
- Choose initial learning rate
- Choose learning rates schedule
- Make some random data
- Train for 100 batches

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from torch.optim.lr_scheduler import StepLR

# define input size, hidden layer size, output size
D_i, D_k, D_o = 10, 40, 5
# create model with two hidden layers
model = nn.Sequential(
    nn.Linear(D_i, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_o))

# He initialization of weights
def weights_init(layer_in):
    if isinstance(layer_in, nn.Linear):
        nn.init.kaiming_uniform(layer_in.weight)
        layer_in.bias.data.fill_(0.0)
model.apply(weights_init)

# choose least squares loss function
criterion = nn.MSELoss()
# construct SGD optimizer and initialize learning rate and momentum
optimizer = torch.optim.SGD(model.parameters(), lr = 0.01, momentum=0.9)
# object that decreases learning rate by half every 10 epochs
scheduler = StepLR(optimizer, step_size=10, gamma=0.5)

# create 100 dummy data points and store in data loader class
x = torch.randn(100, D_i)
y = torch.randn(100, D_o)
data_loader = DataLoader(TensorDataset(x,y), batch_size=10, shuffle=True)

# loop over the dataset 100 times
for epoch in range(100):
    epoch_loss = 0.0
    # loop over batches
    for i, data in enumerate(data_loader):
        # retrieve inputs and labels for this batch
        x_batch, y_batch = data
        # zero the parameter gradients
        optimizer.zero_grad()
        # forward pass
        pred = model(x_batch)
        loss = criterion(pred, y_batch)
        # backward pass
        loss.backward()
        # SGD update
        optimizer.step()
        # update statistics
        epoch_loss += loss.item()
    # print error
    print(f'Epoch {epoch:5d}, loss {epoch_loss:.3f}')
    # tell scheduler to consider updating learning rate
    scheduler.step()
```

PyTorch Code

- Define a neural network
- Initialize parameters
- Define loss function
- Choose optimization algorithm
- Choose initial learning rate
- Choose learning rates schedule
- Make some random data
- Train for 100 batches

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from torch.optim.lr_scheduler import StepLR

# define input size, hidden layer size, output size
D_i, D_k, D_o = 10, 40, 5
# create model with two hidden layers
model = nn.Sequential(
    nn.Linear(D_i, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_o))

# He initialization of weights
def weights_init(layer_in):
    if isinstance(layer_in, nn.Linear):
        nn.init.kaiming_uniform(layer_in.weight)
        layer_in.bias.data.fill_(0.0)
model.apply(weights_init)

# choose least squares loss function
criterion = nn.MSELoss()
# construct SGD optimizer and initialize learning rate and momentum
optimizer = torch.optim.SGD(model.parameters(), lr = 0.01, momentum=0.9)
# object that decreases learning rate by half every 10 epochs
scheduler = StepLR(optimizer, step_size=10, gamma=0.5)

# create 100 dummy data points and store in data loader class
x = torch.randn(100, D_i)
y = torch.randn(100, D_o)
data_loader = DataLoader(TensorDataset(x,y), batch_size=10, shuffle=True)

# loop over the dataset 100 times
for epoch in range(100):
    epoch_loss = 0.0
    # loop over batches
```


PyTorch Code

- Define a neural network
- Initialize parameters
- Define loss function
- Choose optimization algorithm
- Choose initial learning rate
- Choose learning rates schedule
- Make some random data
- Train for 100 batches

Simon J.D. Prince,
"Understanding Deep Learning",
The MIT Press, 2023

```
layer_in.bias.data.fill_(0.0)
model.apply(weights_init)

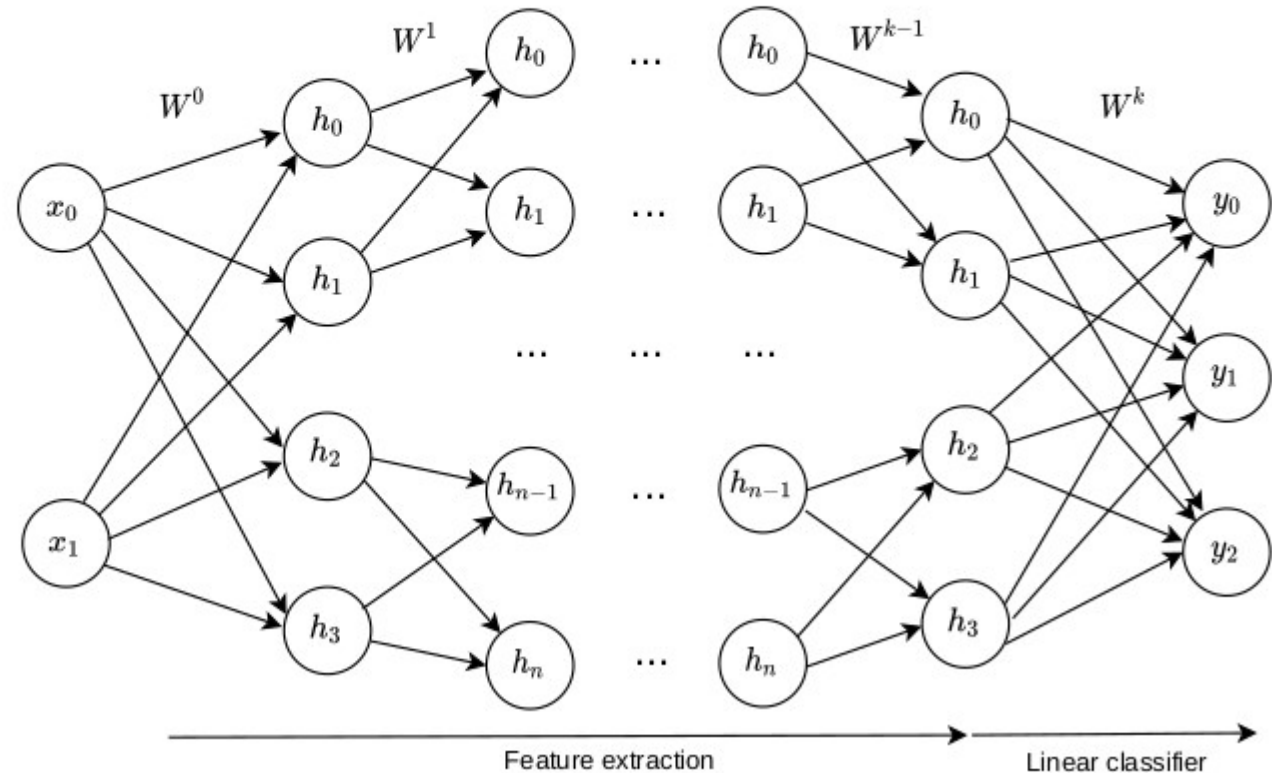
# choose least squares loss function
criterion = nn.MSELoss()
# construct SGD optimizer and initialize learning rate and momentum
optimizer = torch.optim.SGD(model.parameters(), lr = 0.01, momentum=0.9)
# object that decreases learning rate by half every 10 epochs
scheduler = StepLR(optimizer, step_size=10, gamma=0.5)

# create 100 dummy data points and store in data loader class
x = torch.randn(100, D_i)
y = torch.randn(100, D_o)
data_loader = DataLoader(TensorDataset(x,y), batch_size=10, shuffle=True)

# loop over the dataset 100 times
for epoch in range(100):
    epoch_loss = 0.0
    # loop over batches
    for i, data in enumerate(data_loader):
        # retrieve inputs and labels for this batch
        x_batch, y_batch = data
        # zero the parameter gradients
        optimizer.zero_grad()
        # forward pass
        pred = model(x_batch)
        loss = criterion(pred, y_batch)
        # backward pass
        loss.backward()
        # SGD update
        optimizer.step()
        # update statistics
        epoch_loss += loss.item()
    # print error
    print(f'Epoch {epoch:5d}, loss {epoch_loss:.3f}')
    # tell scheduler to consider updating learning rate
    scheduler.step()
```

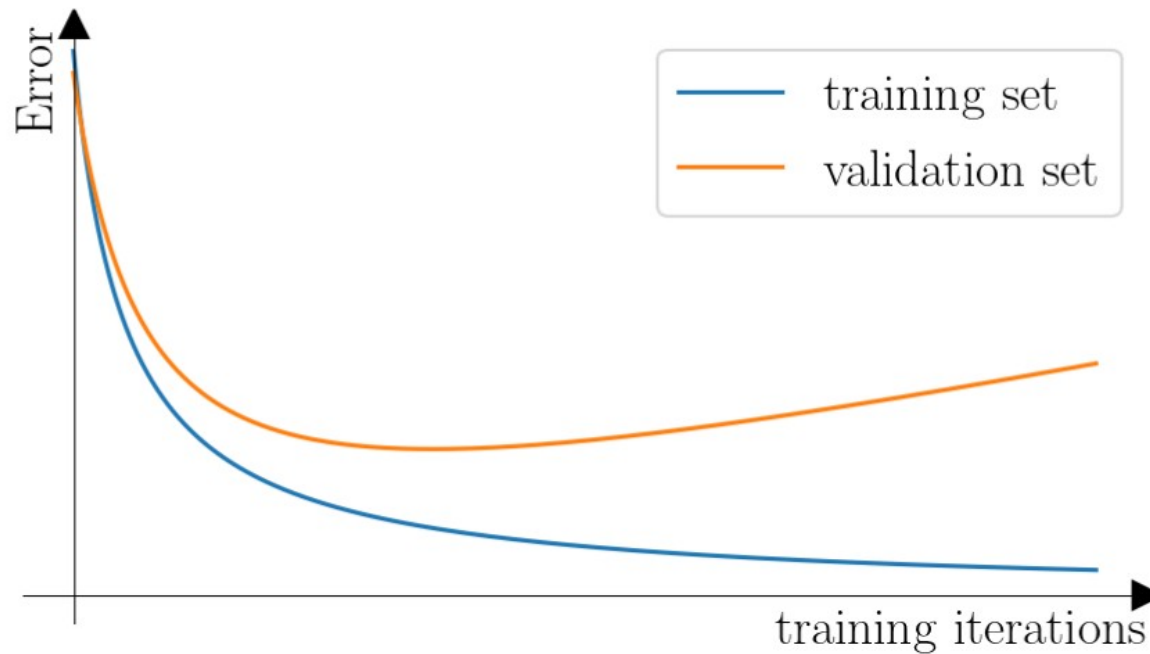
Overfitting

“With great power comes great overfitting” (Joseph Redmon)



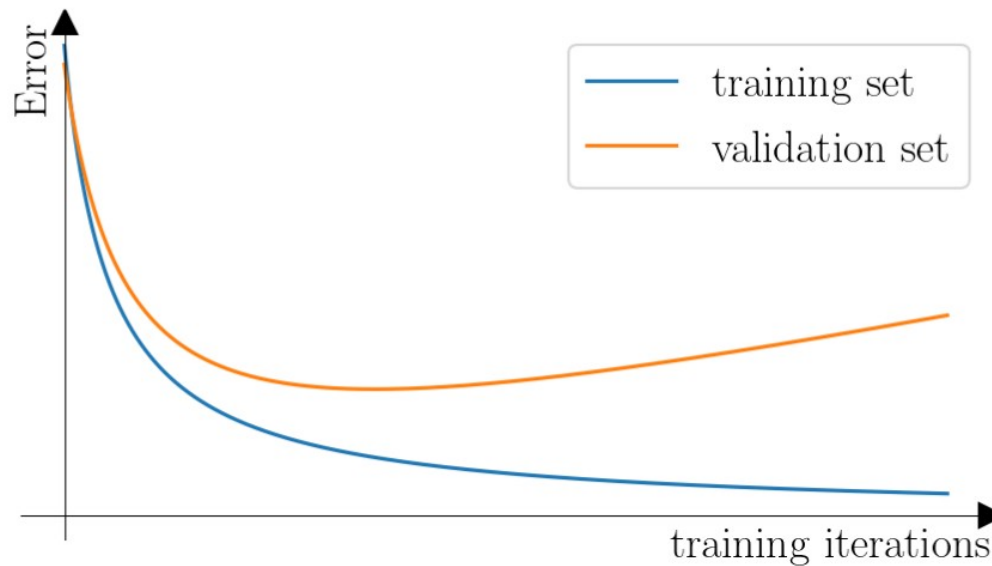
- Deep neural networks have millions of parameters.
- They can learn to fit almost everything. But that's often not what you want.

Bias and Variance in Deep Networks



- A model with high **bias** does not match the training data closely.
- **Variance** refers to changes in the model when using different portions of the training data.
- **Underfitting** occurs when the model is unable to match the input data to the target data. This happens when the model is not complex enough to match all the available data and performs poorly with the training dataset.

Bias and Variance in Deep Networks



- When fitting a model with a gradient descent method, we should always look at the error on training and validation sets over training iteration.
- High bias
→ Increase model complexity
- High variance
→ More data and/or regularization
- In deep learning it's less of a trade-off between bias and variance.
→ We can increase model complexity accompanied with more data/regularization until bias is close to zero, without increasing variance much.

Regularization

- ▶ Early stopping
 - ▶ Weight regularization
 - ▶ Dropout
 - ▶ Data augmentation
 - ▶ Batch normalization
-
- Regularization is very important. Most approaches use multiple of the regularization methods listed here.
 - We will discuss these methods in more detail in the exercises.

Details (How an Image is Analyzed with an ANN) Follow Later

Multiple features
with different kernels

